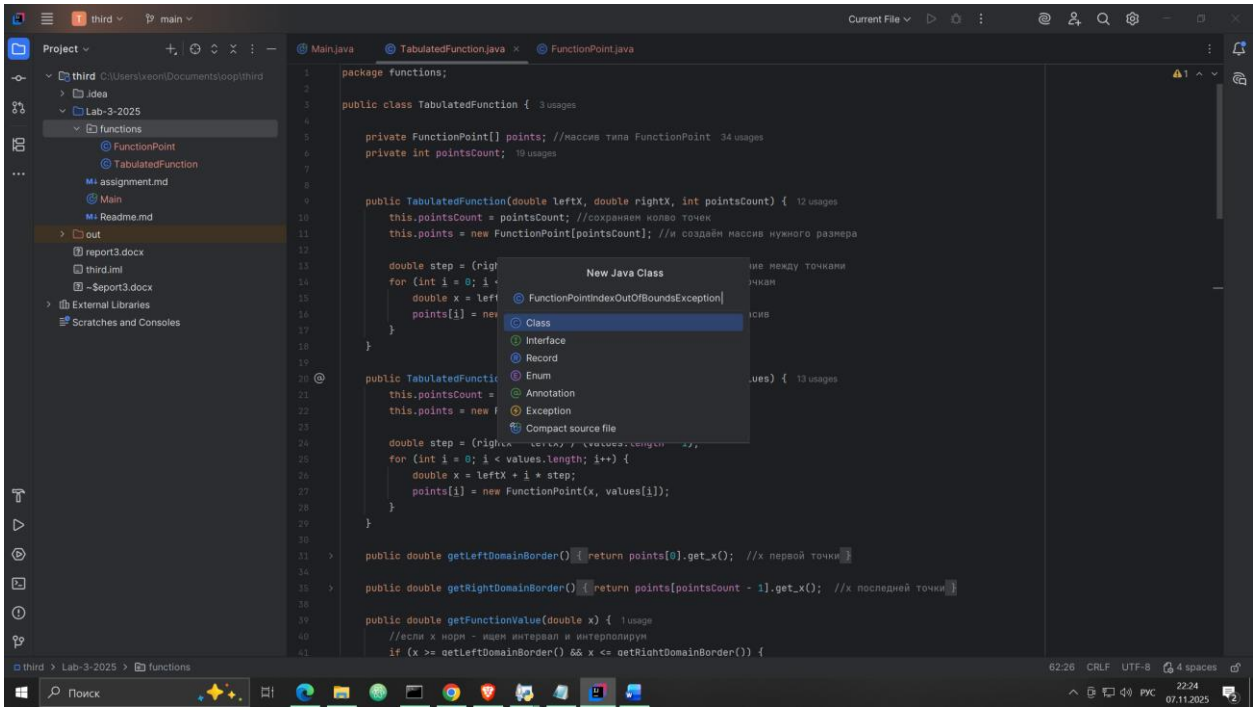
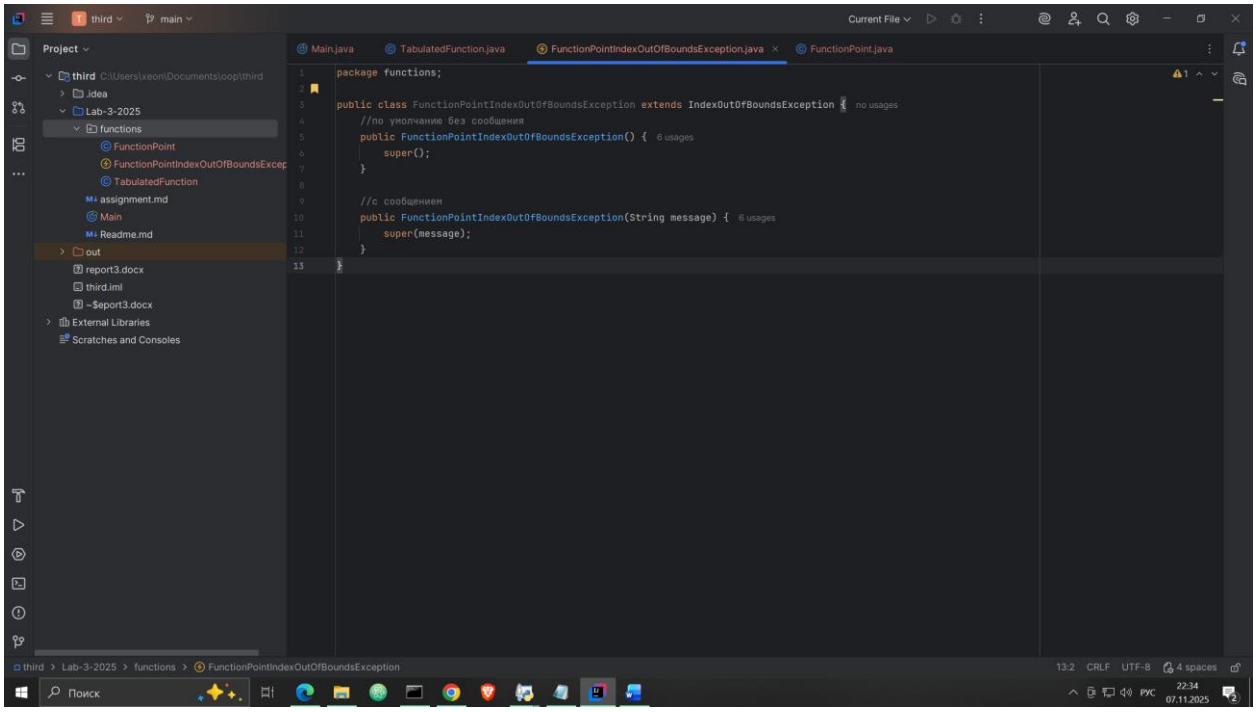
Отчёт по лабораторной работе №3

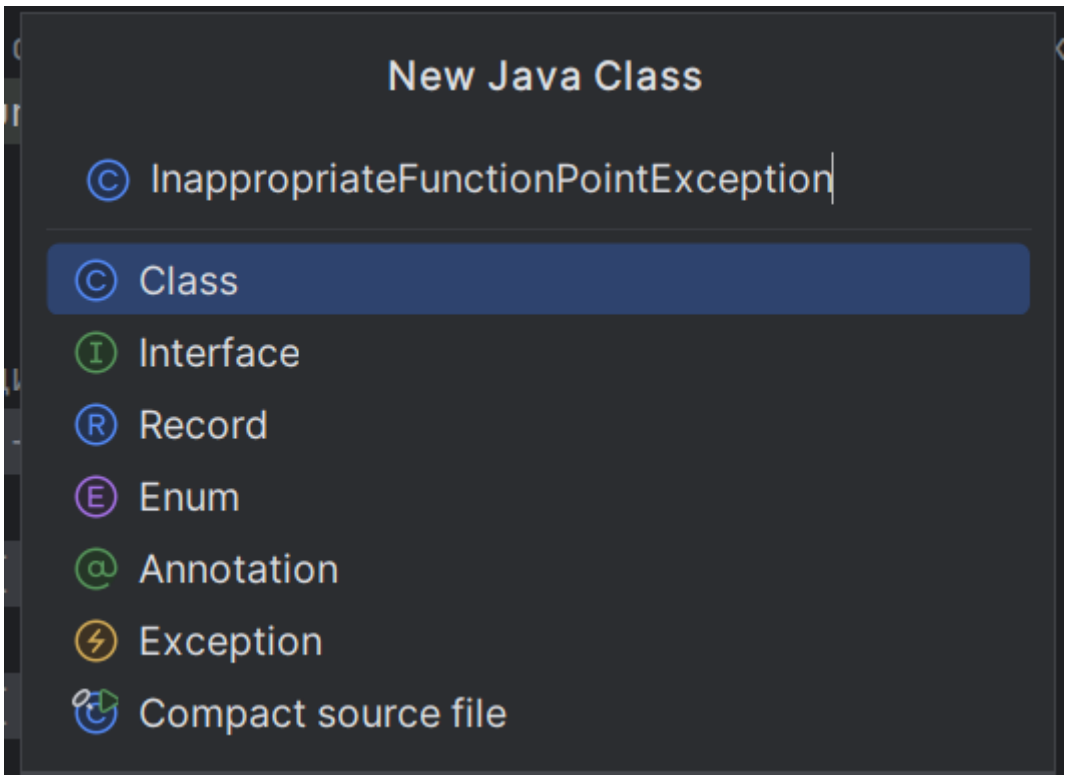
Тенигин Валерий 6204-010302D

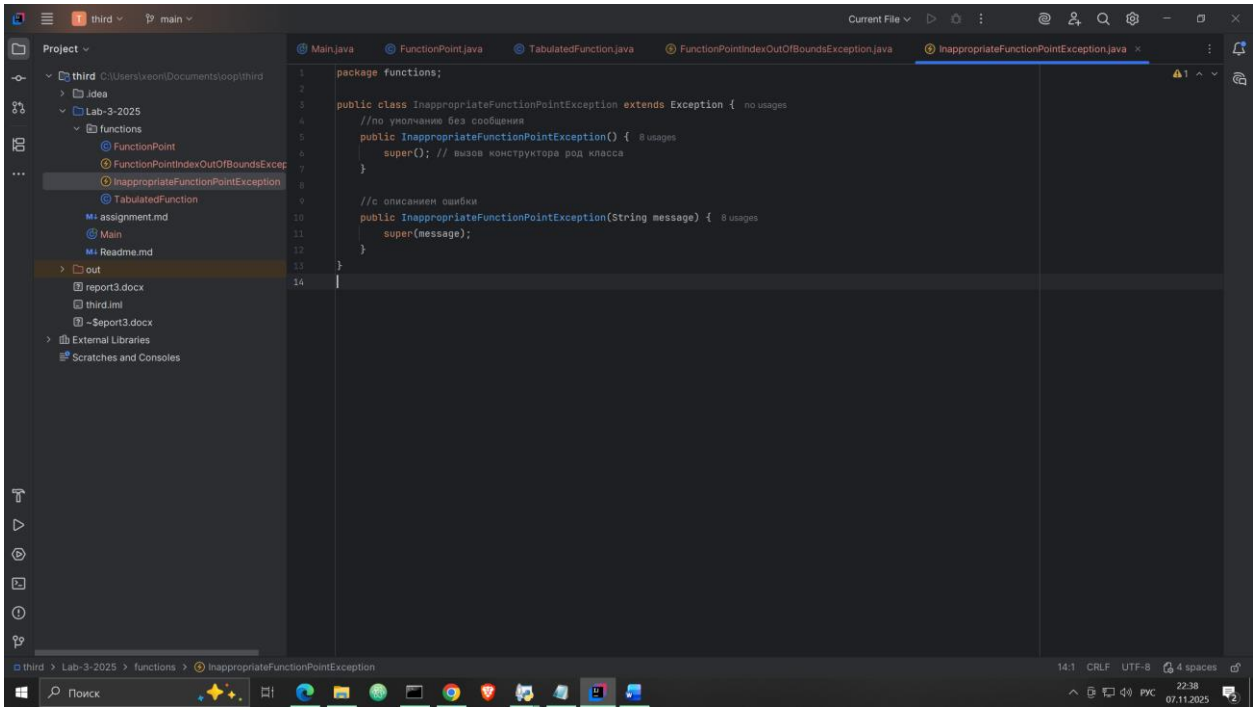
Задание 2

Созданы два класса исключений



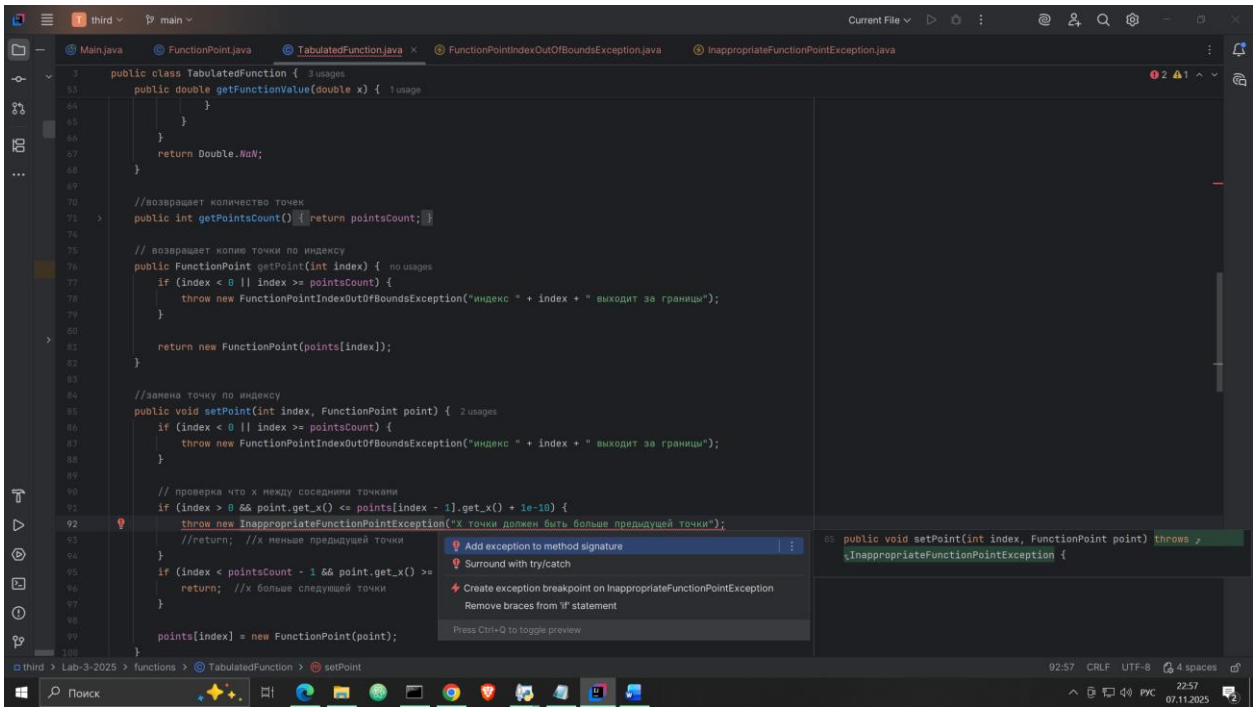


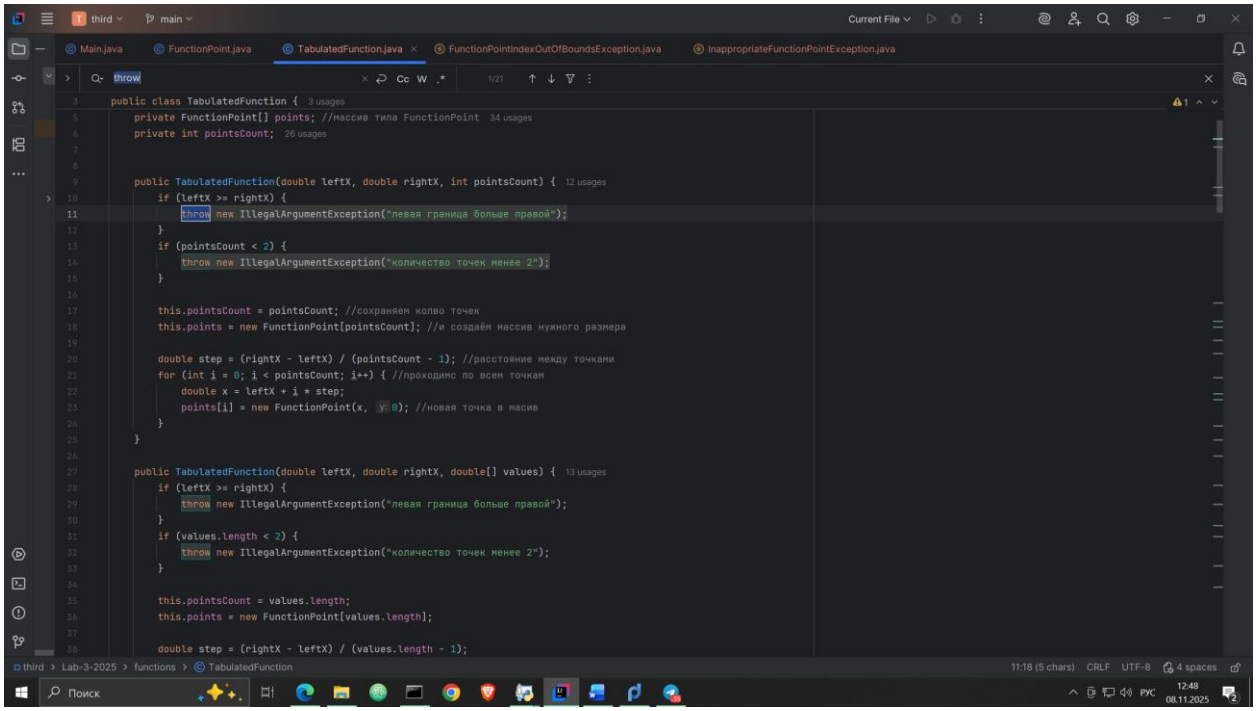

New Java Class
InappropriateFunctionPointException
Class
Interface
Record
Enum
Annotation
Exception
Compact source file

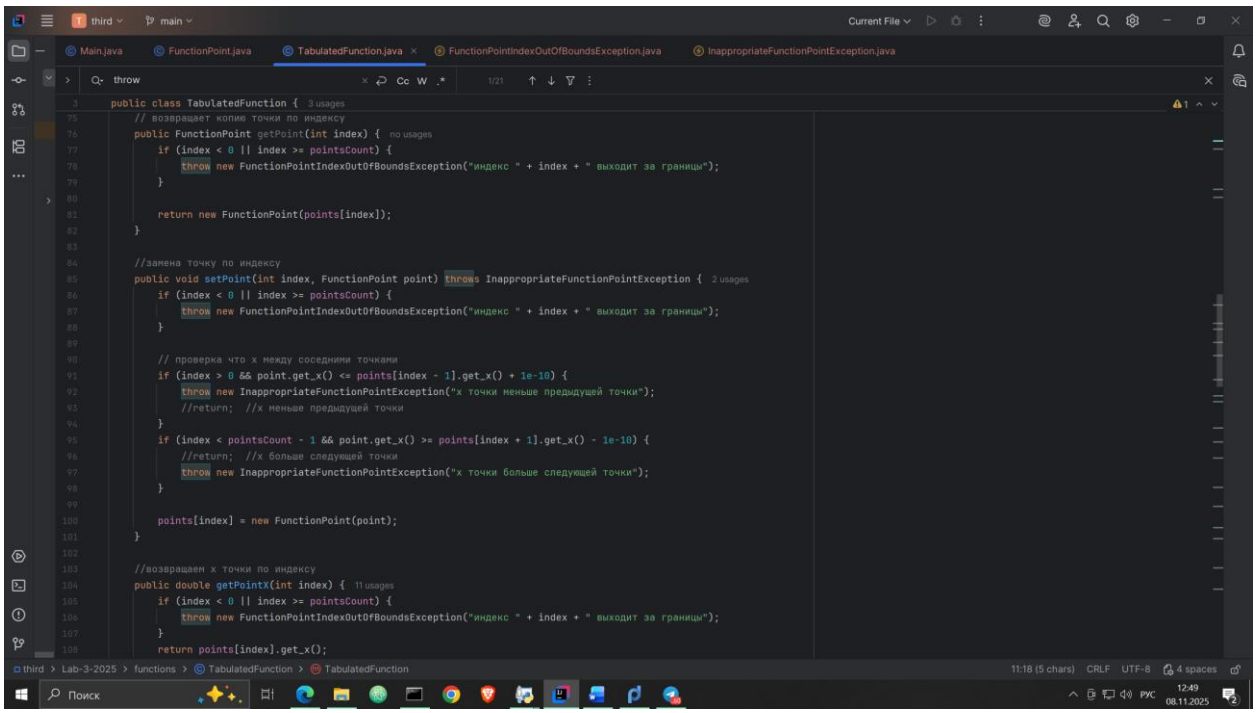

third
main
Current File
Project
Main.java
FunctionPoint.java
TabulatedFunction.java
FunctionPointIndexOutOfBoundsException.java
InappropriateFunctionPointException.java
third C:\Users\xeoni\Documents\oop\third
.idea
Lab-3-2025
functions
FunctionPoint
FunctionPointIndexOutOfBoundsExcep
InappropriateFunctionPointException
TabulatedFunction
assignment.md
Main
Readme.md
out
report3.docx
third.iml
~$eport3.docx
External Libraries
Scratches and Consoles
package functions;
public class InappropriateFunctionPointException extends Exception { no usages
//по умолчанию без сообщения
public InappropriateFunctionPointException() { 8 usages
super(); // вызов конструктора род класса
}
//с описанием ошибки
public InappropriateFunctionPointException(String message) { 8 usages
super(message);
}
}
third › Lab-3-2025 › functions › InappropriateFunctionPointException
14:1 CRLF UTF-8 4 spaces
Поиск
РУС
22:38
07.11.2025

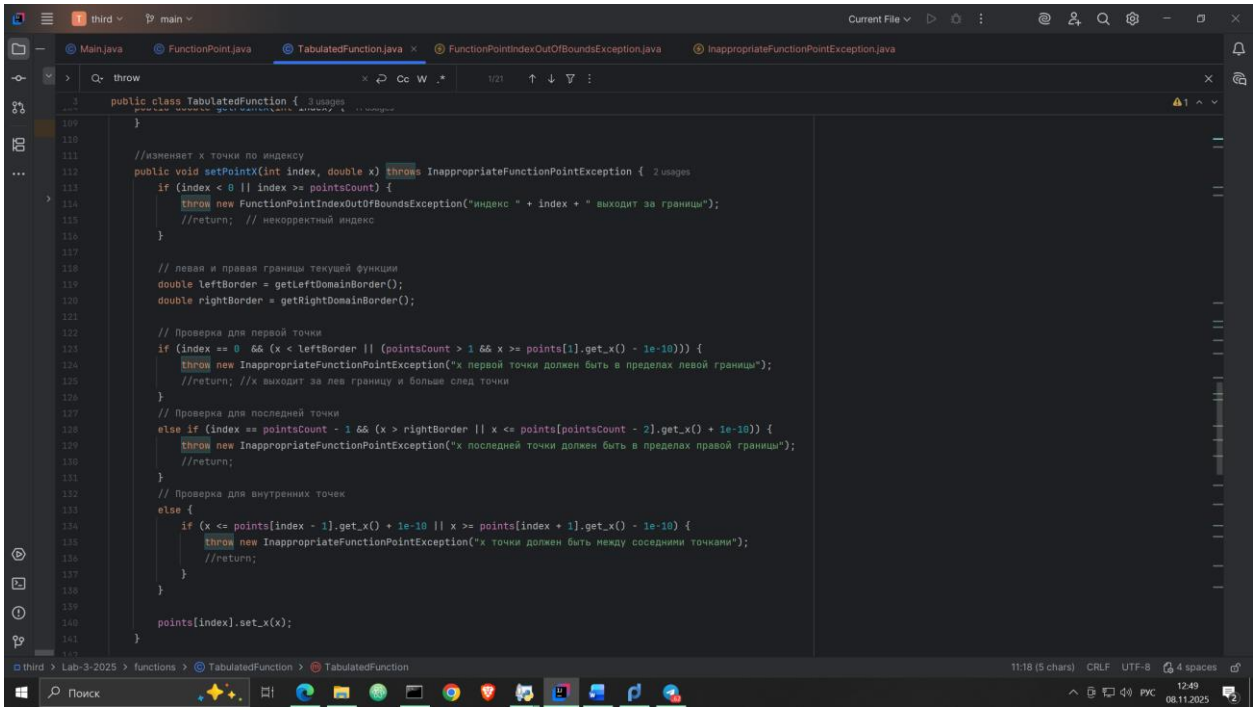
3 номер

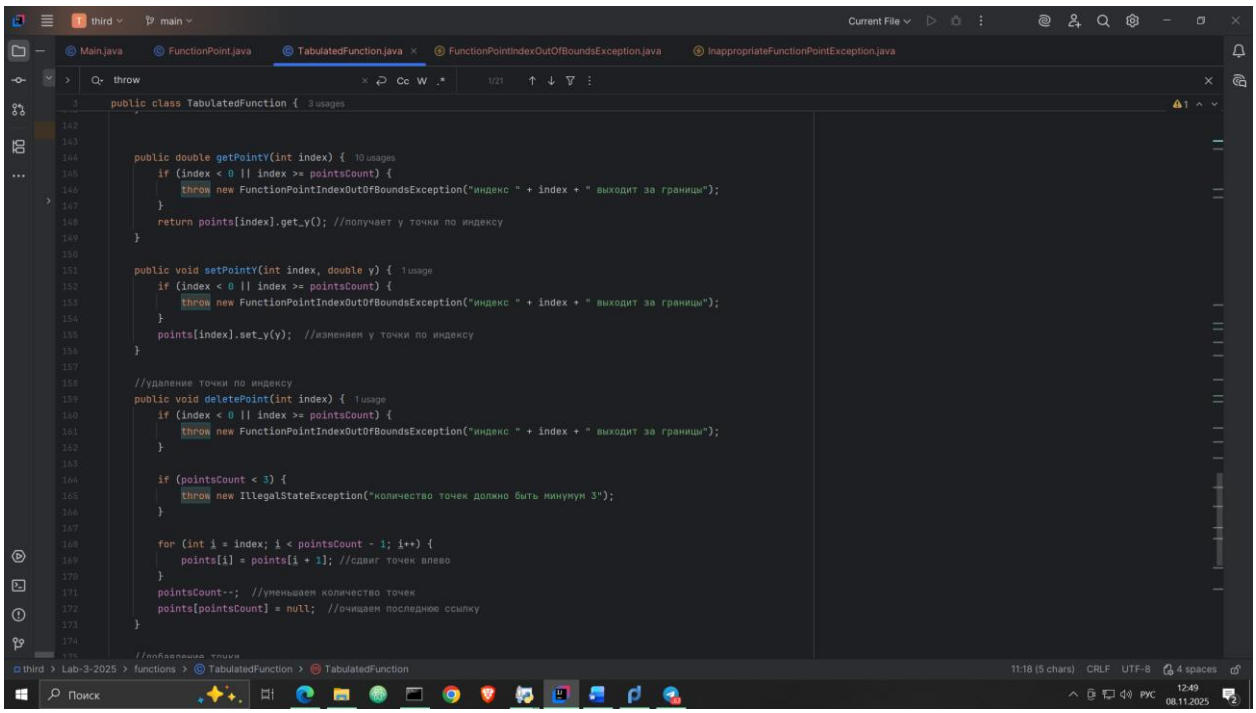
Добавлены проверки исключений, при редактировании кода среда ругалась на изменения, для исправления метод объявлен с throws






third
main
Current File
Main.java
FunctionPoint.java
TabulatedFunction.java
FunctionPointIndexOutOfBoundsException.java
InappropriateFunctionPointException.java
throw
1/21
public class TabulatedFunction { 3 usages
// возвращает копию точки по индексу
public FunctionPoint getPoint(int index) { no usages
if (index < 0 || index >= pointsCount) {
throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
}
return new FunctionPoint(points[index]);
}
//замена точку по индексу
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException { 2 usages
if (index < 0 || index >= pointsCount) {
throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
}
// проверка что x между соседними точками
if (index > 0 && point.get_x() <= points[index - 1].get_x() + 1e-10) {
throw new InappropriateFunctionPointException("x точки меньше предыдущей точки");
//return; //x меньше предыдущей точки
}
if (index < pointsCount - 1 && point.get_x() >= points[index + 1].get_x() - 1e-10) {
//return; //x больше следующей точки
throw new InappropriateFunctionPointException("x точки больше следующей точки");
}
points[index] = new FunctionPoint(point);
}
//возвращаем x точки по индексу
public double getPointX(int index) { 11 usages
if (index < 0 || index >= pointsCount) {
throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
}
return points[index].get_x();
third › Lab-3-2025 › functions › TabulatedFunction › TabulatedFunction
11:18 (5 chars) CRLF UTF-8 4 spaces
Поиск
РУС
12:49
08.11.2025


third
main
Current File
Main.java
FunctionPoint.java
TabulatedFunction.java
FunctionPointIndexOutOfBoundsException.java
InappropriateFunctionPointException.java
throw
1/21
public class TabulatedFunction { 3 usages
}
//изменяет x точки по индексу
public void setPointX(int index, double x) throws InappropriateFunctionPointException { 2 usages
if (index < 0 || index >= pointsCount) {
throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
//return; // некорректный индекс
}
// левая и правая границы текущей функции
double leftBorder = getLeftDomainBorder();
double rightBorder = getRightDomainBorder();
// Проверка для первой точки
if (index == 0 && (x < leftBorder || (pointsCount > 1 && x >= points[1].get_x() - 1e-10))) {
throw new InappropriateFunctionPointException("x первой точки должен быть в пределах левой границы");
//return; //x выходит за лев границу и больше след точки
}
// Проверка для последней точки
else if (index == pointsCount - 1 && (x > rightBorder || x <= points[pointsCount - 2].get_x() + 1e-10)) {
throw new InappropriateFunctionPointException("x последней точки должен быть в пределах правой границы");
//return;
}
// Проверка для внутренних точек
else {
if (x <= points[index - 1].get_x() + 1e-10 || x >= points[index + 1].get_x() - 1e-10) {
throw new InappropriateFunctionPointException("x точки должен быть между соседними точками");
//return;
}
}
points[index].set_x(x);
}
third › Lab-3-2025 › functions › TabulatedFunction › TabulatedFunction
11:18 (5 chars) CRLF UTF-8 4 spaces
Поиск
РУС
12:49
08.11.2025



```
public class TabulatedFunction {  3 usages

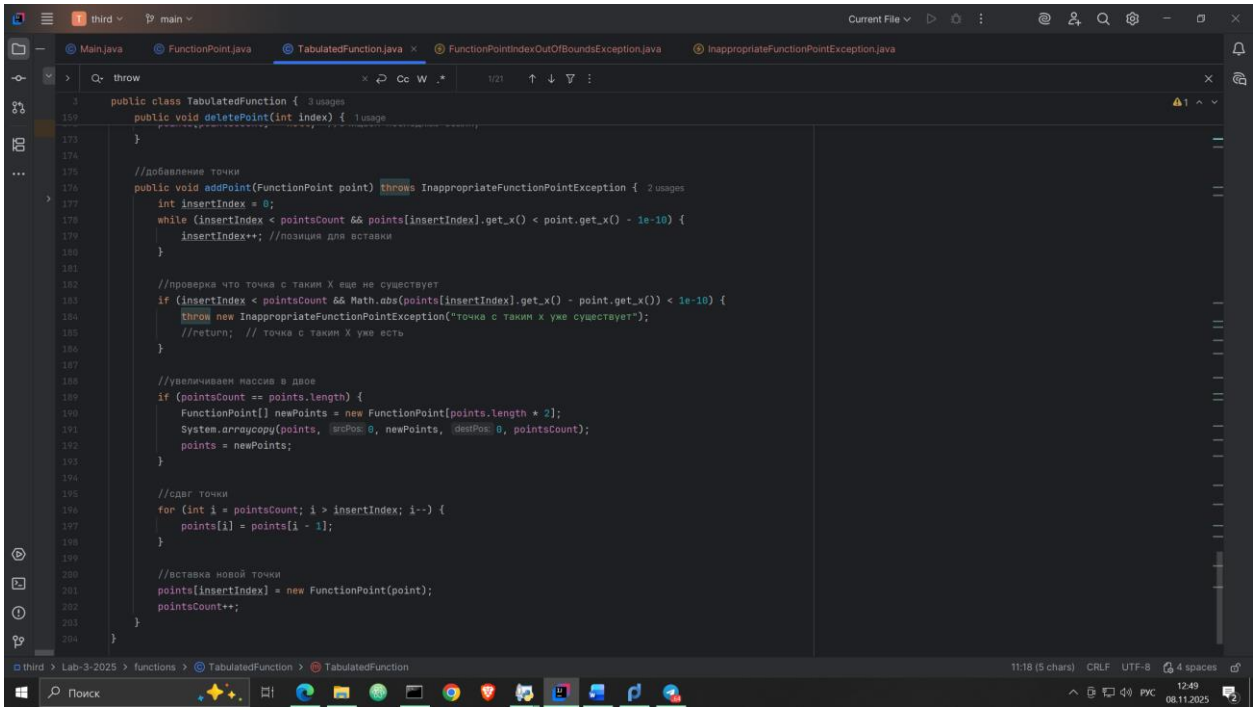
    public double getPointY(int index) {  10 usages
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }
        return points[index].get_y(); //получает у точки по индексу
    }

    public void setPointY(int index, double y) {  1 usage
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }
        points[index].set_y(y);  //изменяем у точки по индексу
    }

    //удаление точки по индексу
    public void deletePoint(int index) {  1 usage
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }

        if (pointsCount < 3) {
            throw new IllegalStateException("количество точек должно быть минумум 3");
        }

        for (int i = index; i < pointsCount - 1; i++) {
            points[i] = points[i + 1]; //сдвиг точек влево
        }
        pointsCount--;  //уменьшаем количество точек
        points[pointsCount] = null;  //очищаем последнюю ссылку
    }
```



```
public class TabulatedFunction {  3 usages
    public void deletePoint(int index) {  1 usage
    }

    //добавление точки
    public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  2 usages
        int insertIndex = 0;
        while (insertIndex < pointsCount && points[insertIndex].get_x() < point.get_x() - 1e-10) {
            insertIndex++; //позиция для вставки
        }

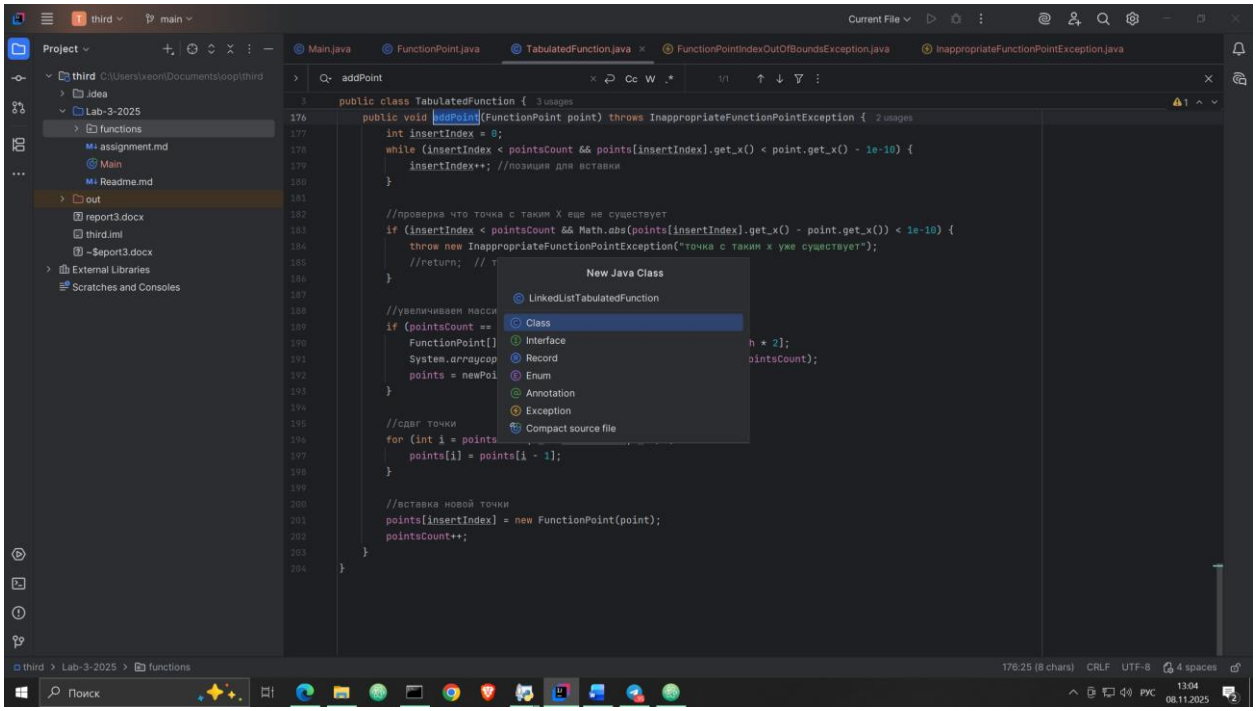
        //проверка что точка с таким X еще не существует
        if (insertIndex < pointsCount && Math.abs(points[insertIndex].get_x() - point.get_x()) < 1e-10) {
            throw new InappropriateFunctionPointException("точка с таким x уже существует");
            //return;  // точка с таким X уже есть
        }

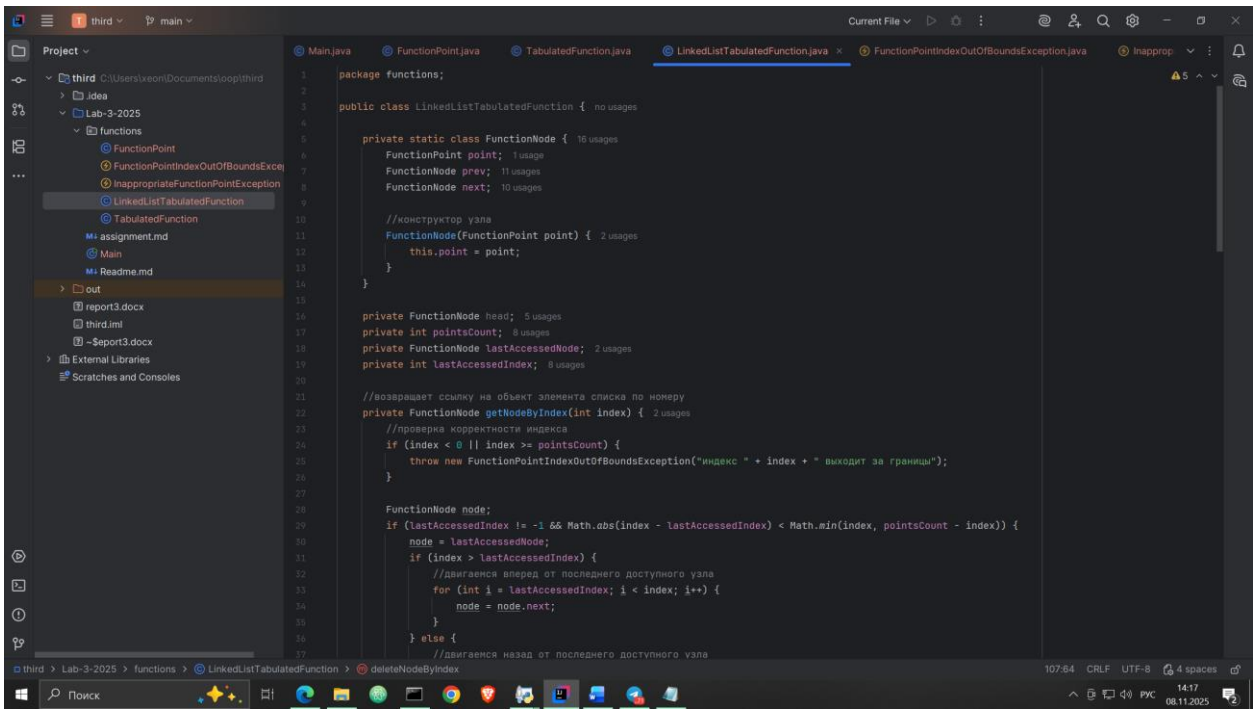
        //увеличиваем массив в двое
        if (pointsCount == points.length) {
            FunctionPoint[] newPoints = new FunctionPoint[points.length * 2];
            System.arraycopy(points,  srcPos: 0, newPoints,  destPos: 0, pointsCount);
            points = newPoints;
        }

        //сдвг точки
        for (int i = pointsCount; i > insertIndex; i--) {
            points[i] = points[i - 1];
        }

        //вставка новой точки
        points[insertIndex] = new FunctionPoint(point);
        pointsCount++;
    }
}
```

Задание 4 - Создан класс на основе связного списка и добавлены основные методы для работы с ним





```java
package functions;

public class LinkedListTabulatedFunction {  no usages

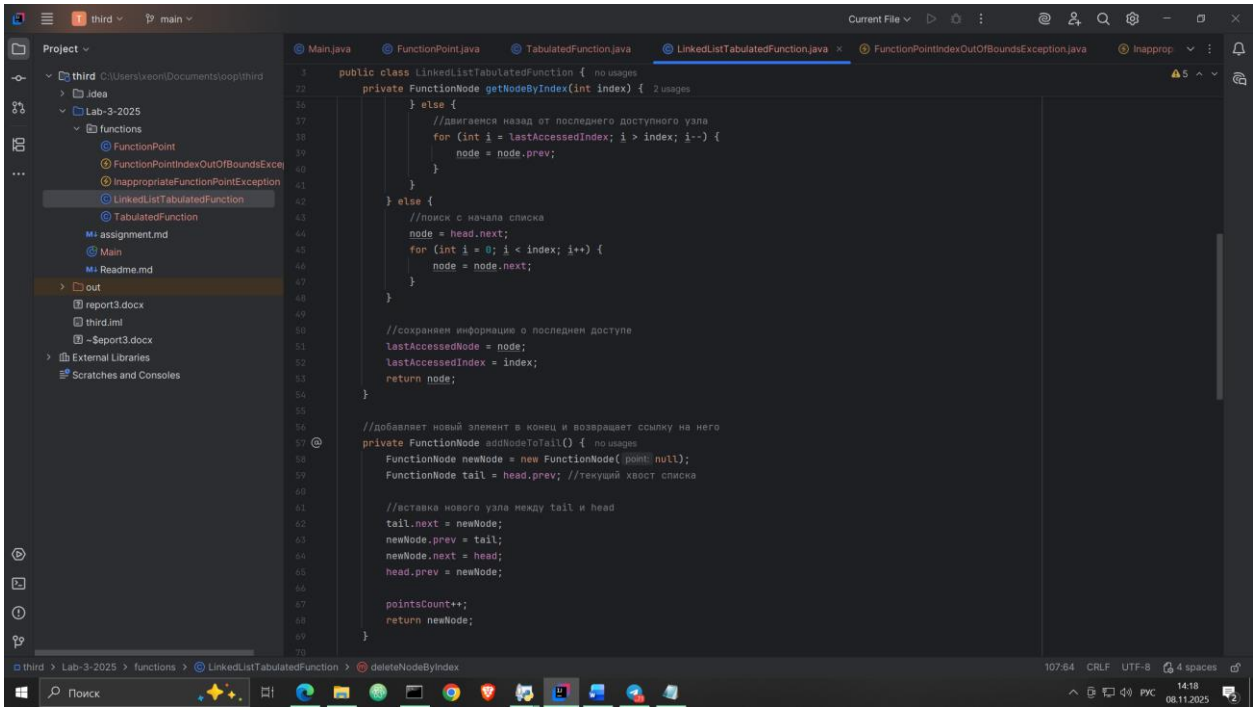
    private static class FunctionNode {  16 usages
        FunctionPoint point;  1 usage
        FunctionNode prev;  11 usages
        FunctionNode next;  10 usages

        //конструктор узла
        FunctionNode(FunctionPoint point) {  2 usages
            this.point = point;
        }
    }

    private FunctionNode head;  5 usages
    private int pointsCount;  8 usages
    private FunctionNode lastAccessedNode;  2 usages
    private int lastAccessedIndex;  8 usages

    //возвращает ссылку на объект элемента списка по номеру
    private FunctionNode getNodeByIndex(int index) {  2 usages
        //проверка корректности индекса
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }

        FunctionNode node;
        if (lastAccessedIndex != -1 && Math.abs(index - lastAccessedIndex) < Math.min(index, pointsCount - index)) {
            node = lastAccessedNode;
            if (index > lastAccessedIndex) {
                //двигаемся вперед от последнего доступного узла
                for (int i = lastAccessedIndex; i < index; i++) {
                    node = node.next;
                }
            } else {
                //двигаемся назад от последнего доступного узла
```



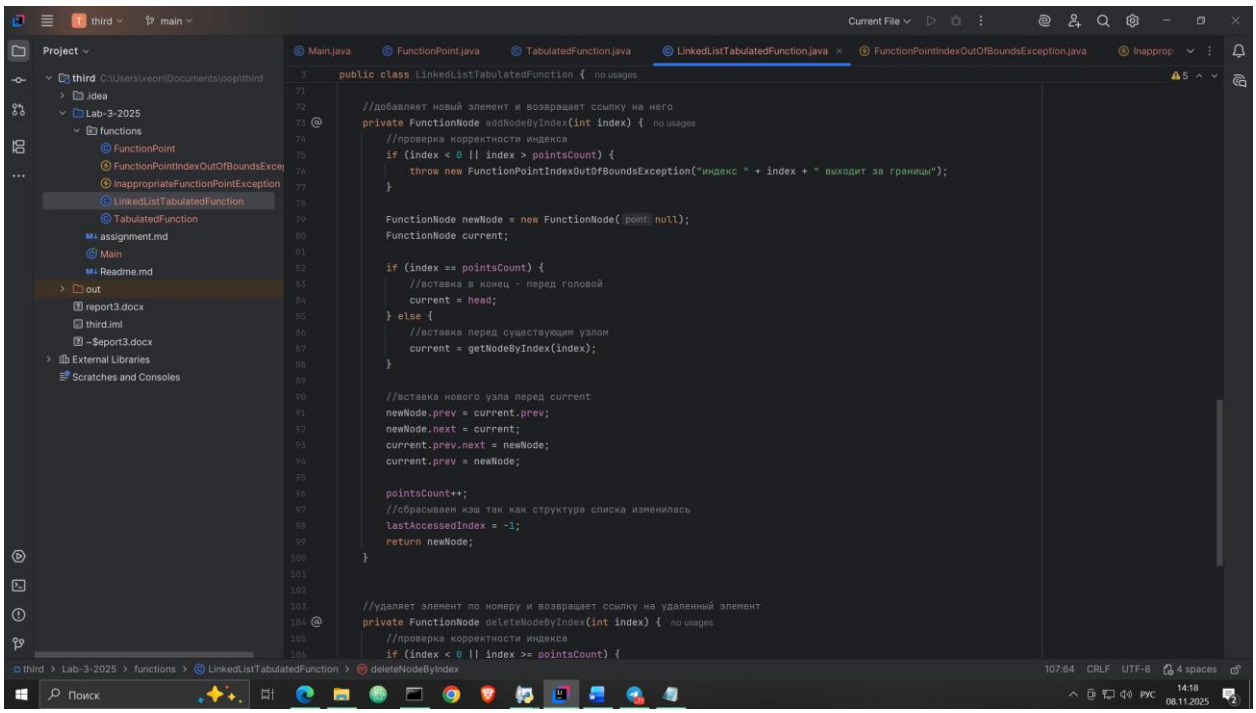
```java
public class LinkedListTabulatedFunction {  no usages
    private FunctionNode getNodeByIndex(int index) {  2 usages
                } else {
                    //двигаемся назад от последнего доступного узла
                    for (int i = lastAccessedIndex; i > index; i--) {
                        node = node.prev;
                    }
                }
            } else {
                //поиск с начала списка
                node = head.next;
                for (int i = 0; i < index; i++) {
                    node = node.next;
                }
            }

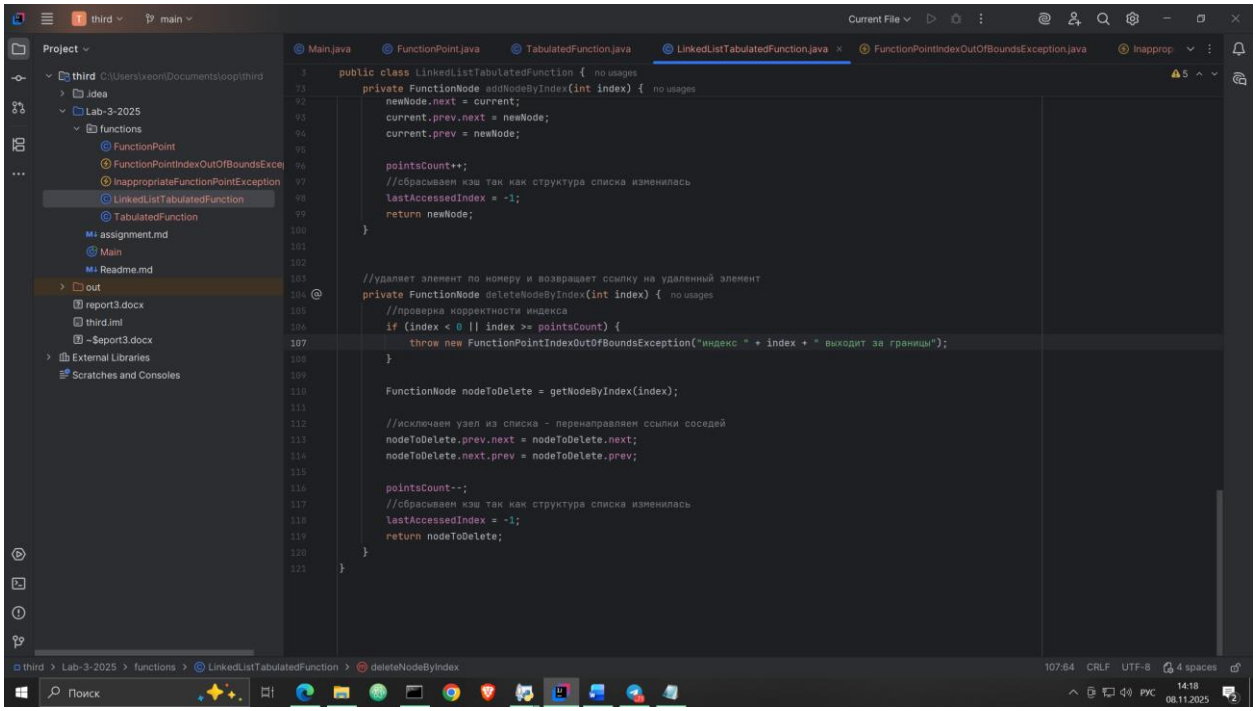
            //сохраняем информацию о последнем доступе
            lastAccessedNode = node;
            lastAccessedIndex = index;
            return node;
        }

        //добавляет новый элемент в конец и возвращает ссылку на него
        private FunctionNode addNodeToTail() {  no usages
            FunctionNode newNode = new FunctionNode( point: null);
            FunctionNode tail = head.prev; //текущий хвост списка

            //вставка нового узла между tail и head
            tail.next = newNode;
            newNode.prev = tail;
            newNode.next = head;
            head.prev = newNode;

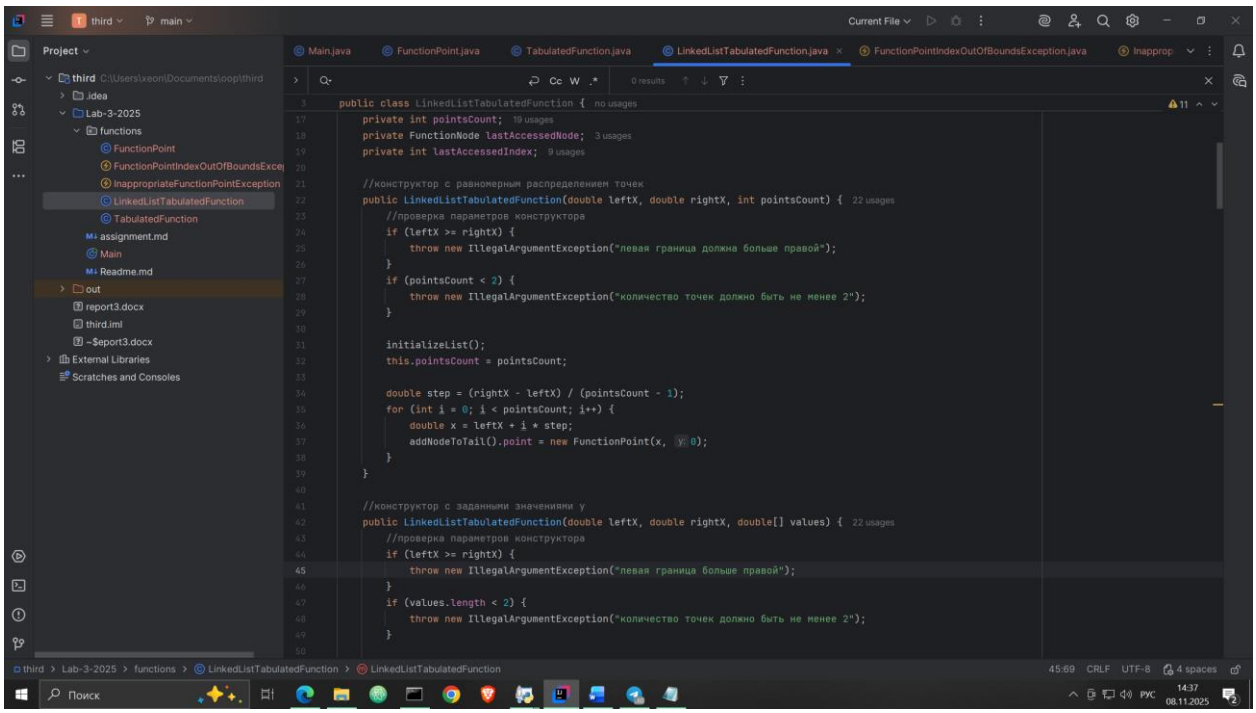
            pointsCount++;
            return newNode;
        }
```

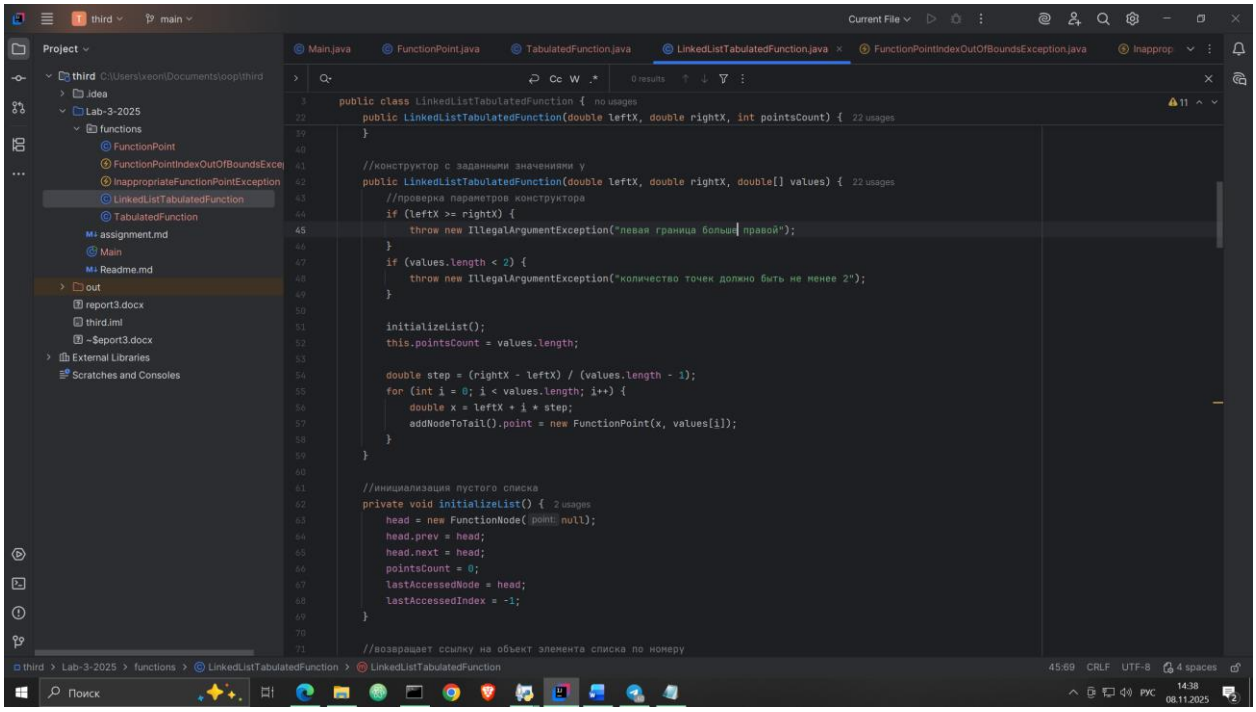


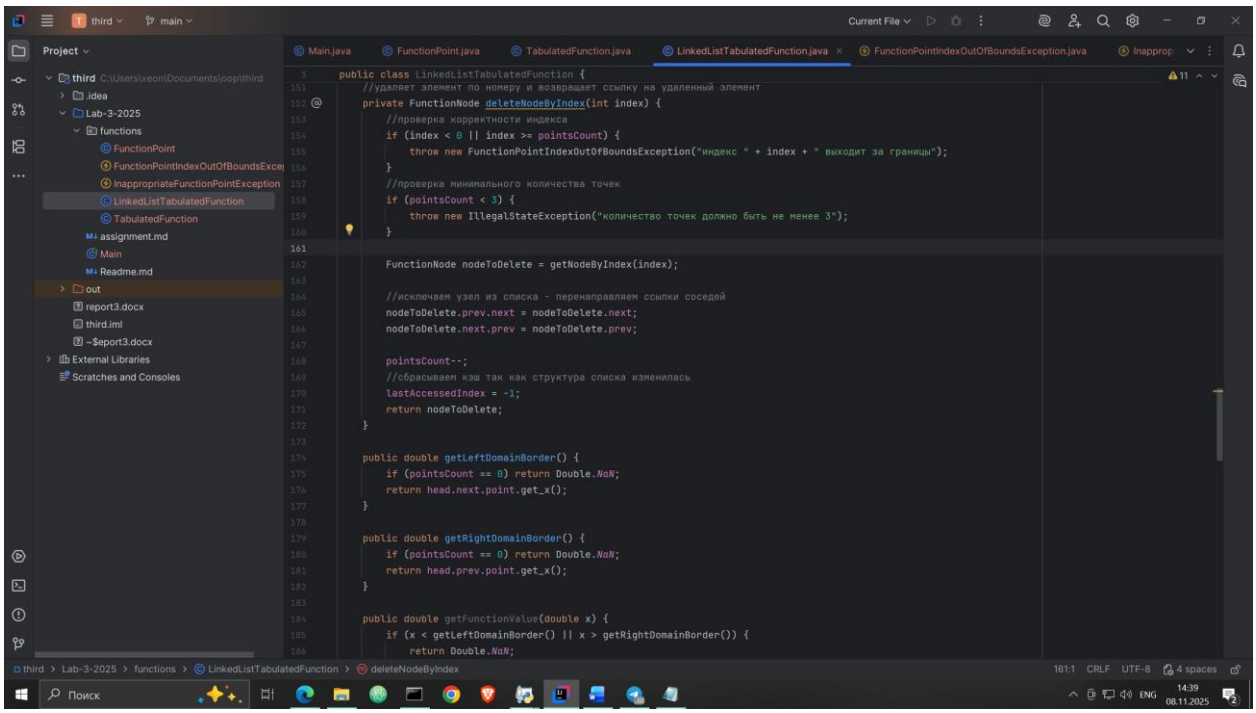


Задание 5

Добавлены конструкторы, метод инициализации списка, доп проверка в deleteNodeByIndex и все методы табулированной функции


third
main
Current File
Project
Main.java
FunctionPoint.java
TabulatedFunction.java
LinkedListTabulatedFunction.java
FunctionPointIndexOutOfBoundsException.java
third C:\Users\xeoni\Documents\oop\third
.idea
Lab-3-2025
functions
FunctionPoint
FunctionPointIndexOutOfBoundsExcep
InappropriateFunctionPointException
LinkedListTabulatedFunction
TabulatedFunction
assignment.md
Main
Readme.md
out
report3.docx
third.iml
~$eport3.docx
External Libraries
Scratches and Consoles
public class LinkedListTabulatedFunction { no usages
private int pointsCount; 19 usages
private FunctionNode lastAccessedNode; 3 usages
private int lastAccessedIndex; 9 usages
//конструктор с равномерным распределением точек
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) { 22 usages
//проверка параметров конструктора
if (leftX >= rightX) {
throw new IllegalArgumentException("левая граница должна больше правой");
}
if (pointsCount < 2) {
throw new IllegalArgumentException("количество точек должно быть не менее 2");
}
initializeList();
this.pointsCount = pointsCount;
double step = (rightX - leftX) / (pointsCount - 1);
for (int i = 0; i < pointsCount; i++) {
double x = leftX + i * step;
addNodeToTail().point = new FunctionPoint(x, y: 0);
}
}
//конструктор с заданными значениями y
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) { 22 usages
//проверка параметров конструктора
if (leftX >= rightX) {
throw new IllegalArgumentException("левая граница больше правой");
}
if (values.length < 2) {
throw new IllegalArgumentException("количество точек должно быть не менее 2");
}
third > Lab-3-2025 > functions > LinkedListTabulatedFunction > LinkedListTabulatedFunction
45:69 CRLF UTF-8 4 spaces
Поиск
РУС
14:37
08.11.2025


third
main
Current File
Project
Main.java
FunctionPoint.java
TabulatedFunction.java
LinkedListTabulatedFunction.java
FunctionPointIndexOutOfBoundsException.java
third C:\Users\xeoni\Documents\oop\third
.idea
Lab-3-2025
functions
FunctionPoint
FunctionPointIndexOutOfBoundsExcep
InappropriateFunctionPointException
LinkedListTabulatedFunction
TabulatedFunction
assignment.md
Main
Readme.md
out
report3.docx
third.iml
~$eport3.docx
External Libraries
Scratches and Consoles
public class LinkedListTabulatedFunction { no usages
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) { 22 usages
}
//конструктор с заданными значениями y
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) { 22 usages
//проверка параметров конструктора
if (leftX >= rightX) {
throw new IllegalArgumentException("левая граница больше правой");
}
if (values.length < 2) {
throw new IllegalArgumentException("количество точек должно быть не менее 2");
}
initializeList();
this.pointsCount = values.length;
double step = (rightX - leftX) / (values.length - 1);
for (int i = 0; i < values.length; i++) {
double x = leftX + i * step;
addNodeToTail().point = new FunctionPoint(x, values[i]);
}
}
//инициализация пустого списка
private void initializeList() { 2 usages
head = new FunctionNode(point: null);
head.prev = head;
head.next = head;
pointsCount = 0;
lastAccessedNode = head;
lastAccessedIndex = -1;
}
//возвращает ссылку на объект элемента списка по номеру
third > Lab-3-2025 > functions > LinkedListTabulatedFunction > LinkedListTabulatedFunction
45:69 CRLF UTF-8 4 spaces
Поиск
РУС
14:38
08.11.2025



```
public class LinkedListTabulatedFunction {
    //удаляет элемент по номеру и возвращает ссылку на удаленный элемент
    private FunctionNode deleteNodeByIndex(int index) {
        //проверка корректности индекса
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }
        //проверка минимального количества точек
        if (pointsCount < 3) {
            throw new IllegalStateException("количество точек должно быть не менее 3");
        }

        FunctionNode nodeToDelete = getNodeByIndex(index);

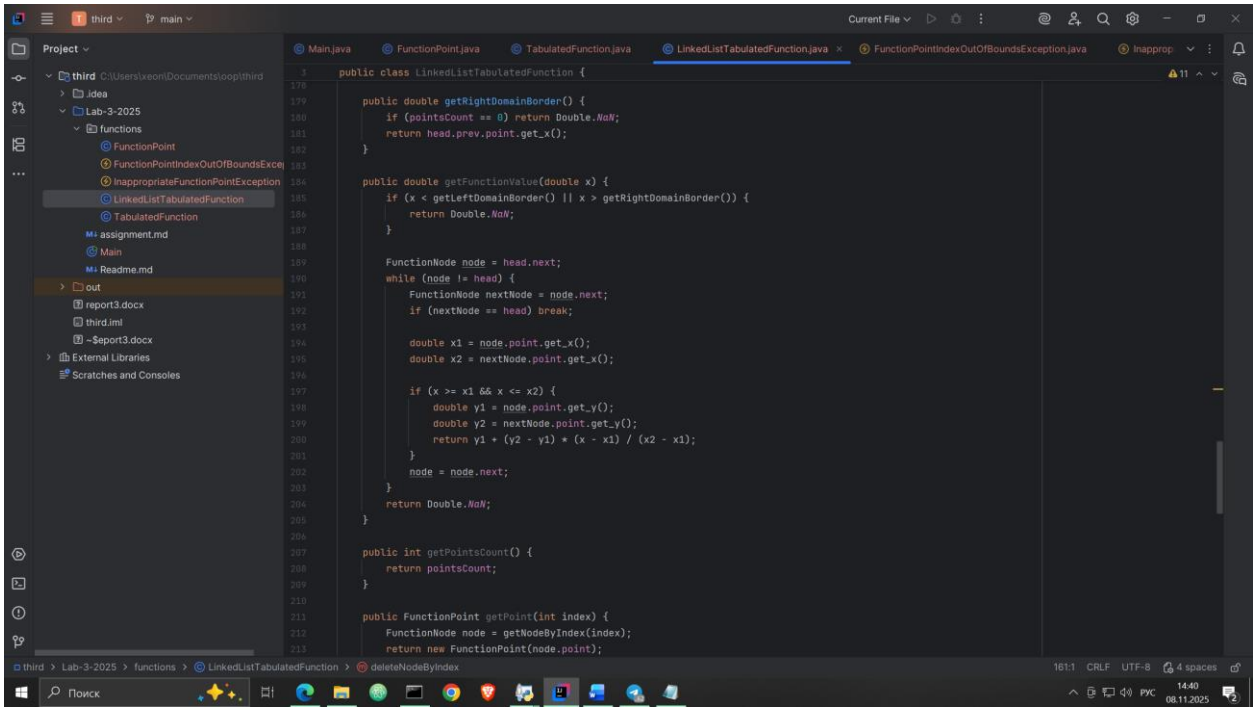
        //исключаем узел из списка - перенаправляем ссылки соседей
        nodeToDelete.prev.next = nodeToDelete.next;
        nodeToDelete.next.prev = nodeToDelete.prev;

        pointsCount--;
        //сбрасываем кэш так как структура списка изменилась
        lastAccessedIndex = -1;
        return nodeToDelete;
    }

    public double getLeftDomainBorder() {
        if (pointsCount == 0) return Double.NaN;
        return head.next.point.get_x();
    }

    public double getRightDomainBorder() {
        if (pointsCount == 0) return Double.NaN;
        return head.prev.point.get_x();
    }

    public double getFunctionValue(double x) {
        if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
            return Double.NaN;
```



```
public class LinkedListTabulatedFunction {

    public double getRightDomainBorder() {
        if (pointsCount == 0) return Double.NaN;
        return head.prev.point.get_x();
    }

    public double getFunctionValue(double x) {
        if (x < getLeftDomainBorder() || x > getRightDomainBorder()) {
            return Double.NaN;
        }

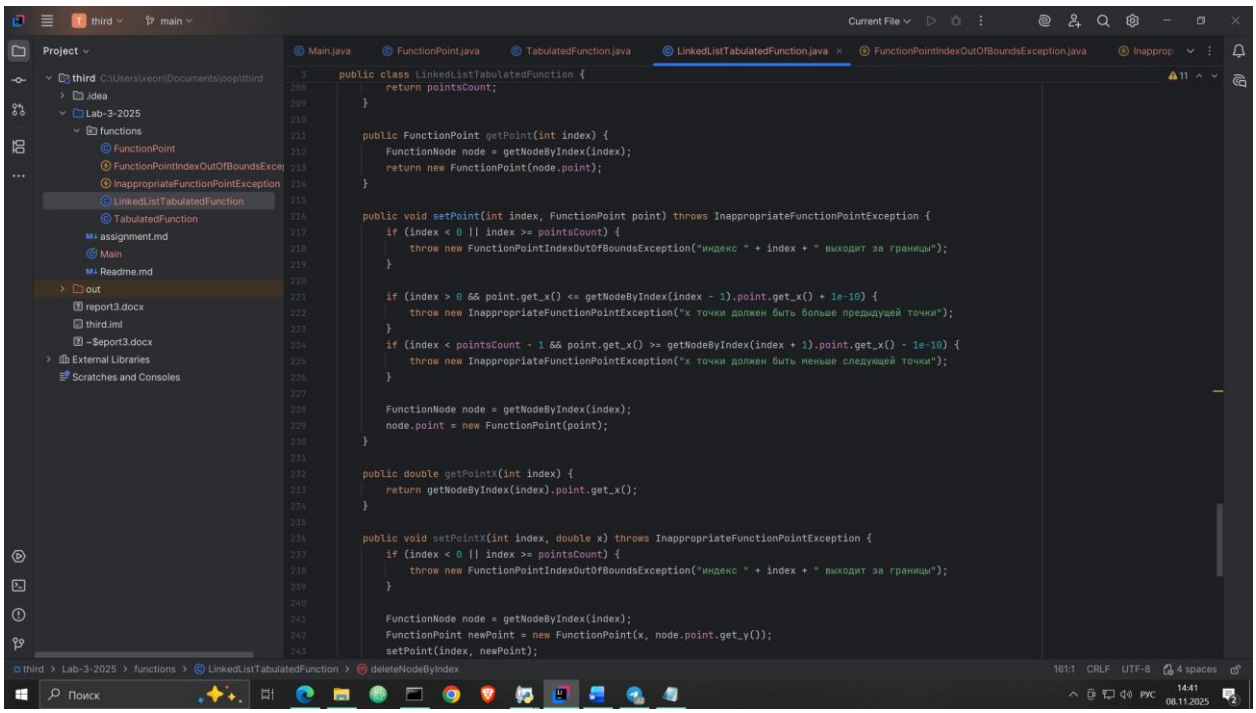
        FunctionNode node = head.next;
        while (node != head) {
            FunctionNode nextNode = node.next;
            if (nextNode == head) break;

            double x1 = node.point.get_x();
            double x2 = nextNode.point.get_x();

            if (x >= x1 && x <= x2) {
                double y1 = node.point.get_y();
                double y2 = nextNode.point.get_y();
                return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
            }
            node = node.next;
        }
        return Double.NaN;
    }

    public int getPointsCount() {
        return pointsCount;
    }

    public FunctionPoint getPoint(int index) {
        FunctionNode node = getNodeByIndex(index);
        return new FunctionPoint(node.point);
```



```
public class LinkedListTabulatedFunction {
        return pointsCount;
    }

    public FunctionPoint getPoint(int index) {
        FunctionNode node = getNodeByIndex(index);
        return new FunctionPoint(node.point);
    }

    public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }

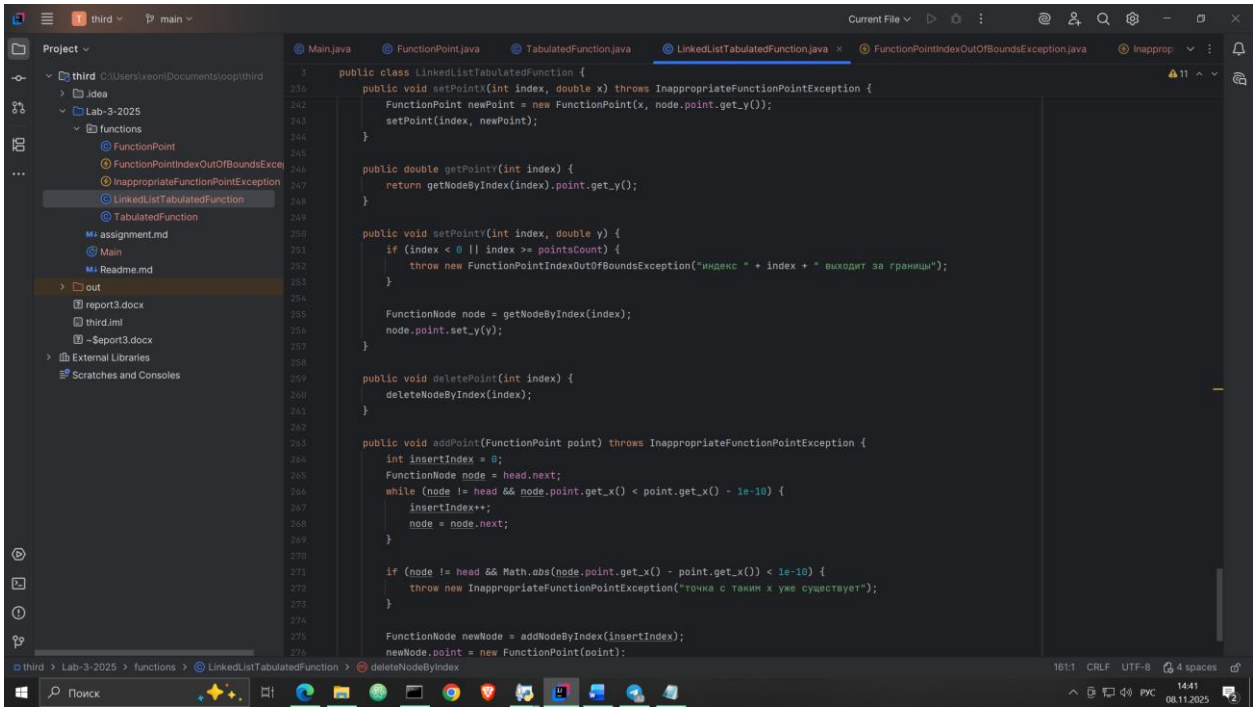
        if (index > 0 && point.get_x() <= getNodeByIndex(index - 1).point.get_x() + 1e-10) {
            throw new InappropriateFunctionPointException("x точки должен быть больше предыдущей точки");
        }
        if (index < pointsCount - 1 && point.get_x() >= getNodeByIndex(index + 1).point.get_x() - 1e-10) {
            throw new InappropriateFunctionPointException("x точки должен быть меньше следующей точки");
        }

        FunctionNode node = getNodeByIndex(index);
        node.point = new FunctionPoint(point);
    }

    public double getPointX(int index) {
        return getNodeByIndex(index).point.get_x();
    }

    public void setPointX(int index, double x) throws InappropriateFunctionPointException {
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }

        FunctionNode node = getNodeByIndex(index);
        FunctionPoint newPoint = new FunctionPoint(x, node.point.get_y());
        setPoint(index, newPoint);
```



```
public class LinkedListTabulatedFunction {
    public void setPointX(int index, double x) throws InappropriateFunctionPointException {
        FunctionPoint newPoint = new FunctionPoint(x, node.point.get_y());
        setPoint(index, newPoint);
    }

    public double getPointY(int index) {
        return getNodeByIndex(index).point.get_y();
    }

    public void setPointY(int index, double y) {
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("индекс " + index + " выходит за границы");
        }

        FunctionNode node = getNodeByIndex(index);
        node.point.set_y(y);
    }

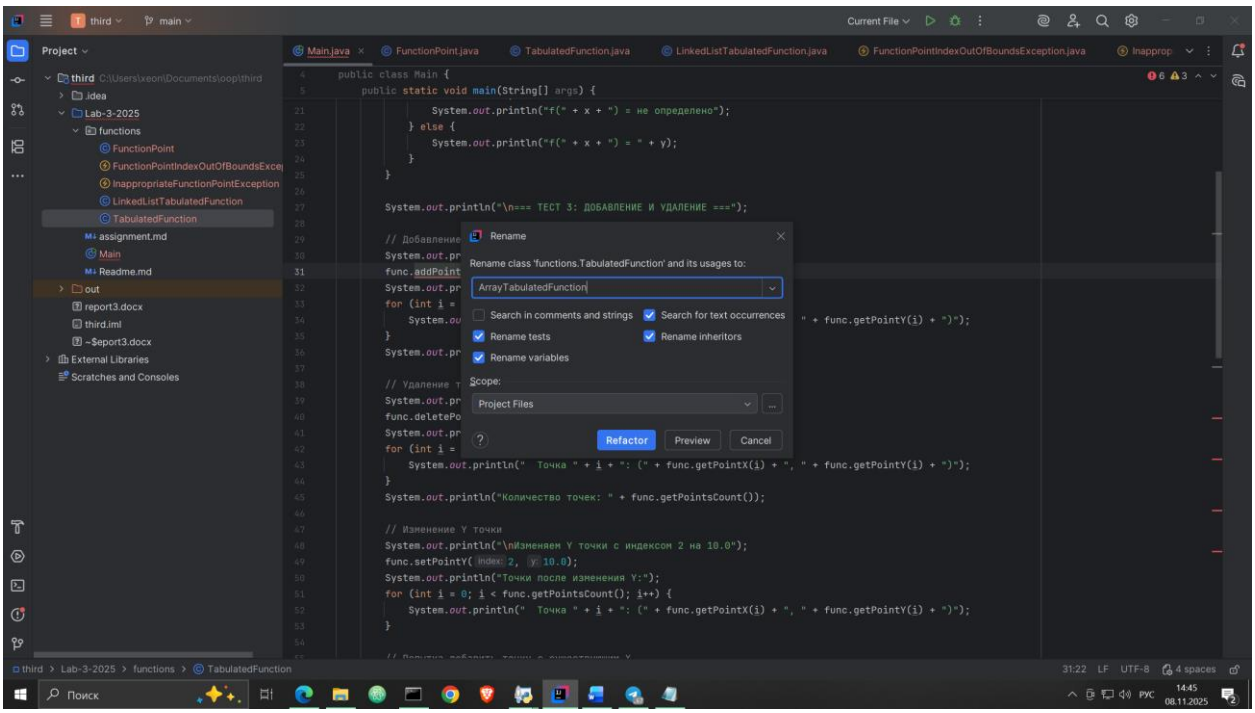
    public void deletePoint(int index) {
        deleteNodeByIndex(index);
    }

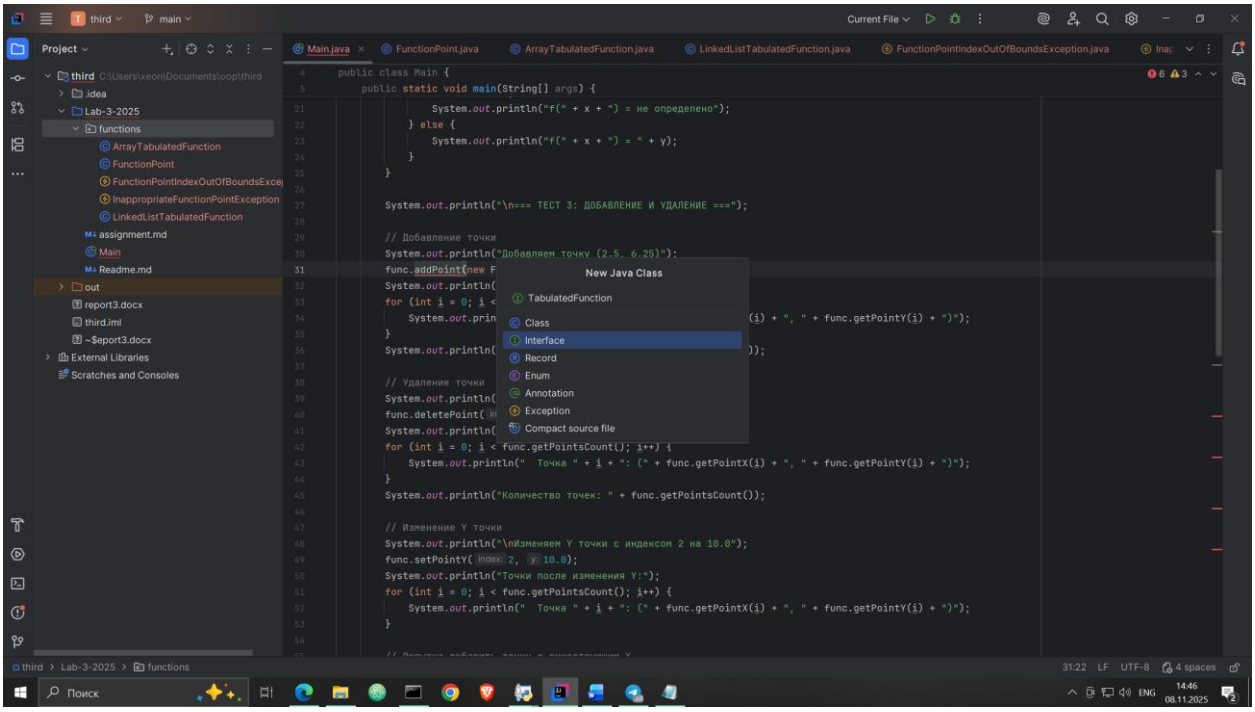
    public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
        int insertIndex = 0;
        FunctionNode node = head.next;
        while (node != head && node.point.get_x() < point.get_x() - 1e-10) {
            insertIndex++;
            node = node.next;
        }

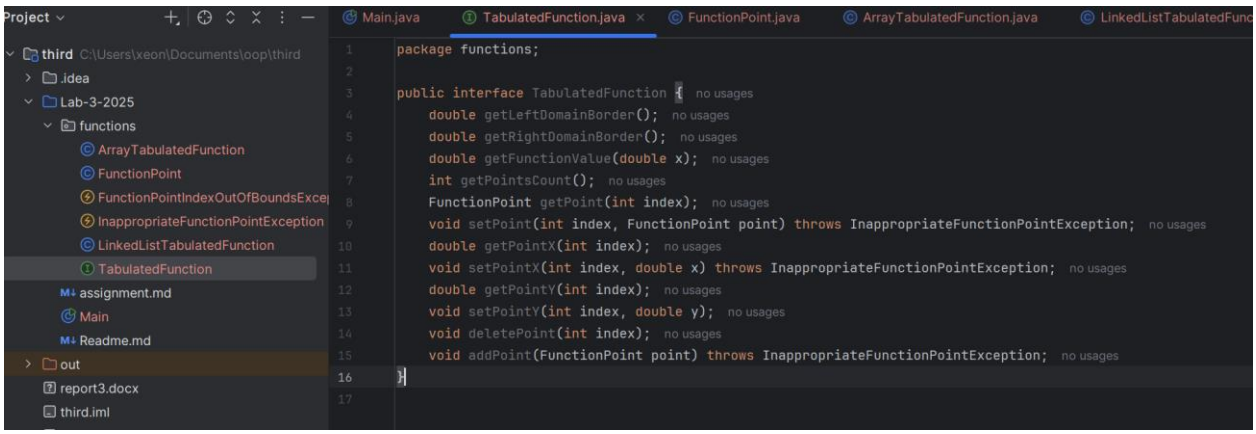
        if (node != head && Math.abs(node.point.get_x() - point.get_x()) < 1e-10) {
            throw new InappropriateFunctionPointException("точка с таким x уже существует");
        }

        FunctionNode newNode = addNodeByIndex(insertIndex);
        newNode.point = new FunctionPoint(point);
```

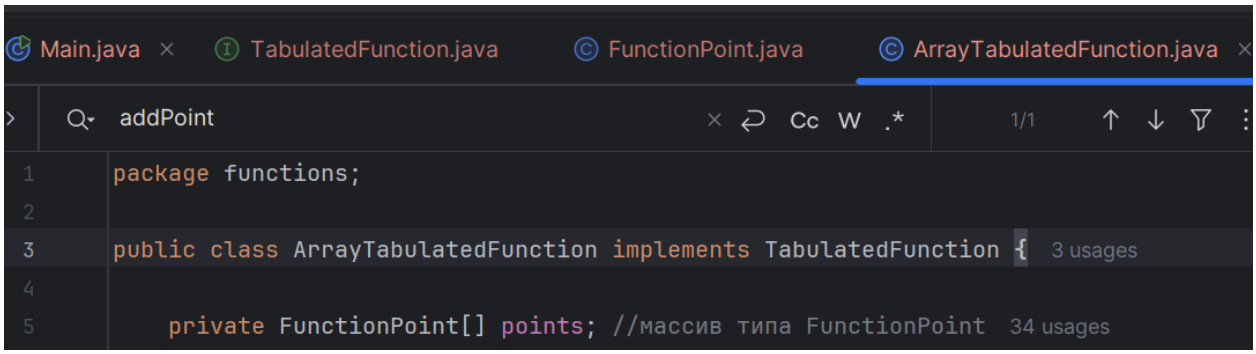
Задание 6 – согласно заданию был переименован класс и создан интерфейс

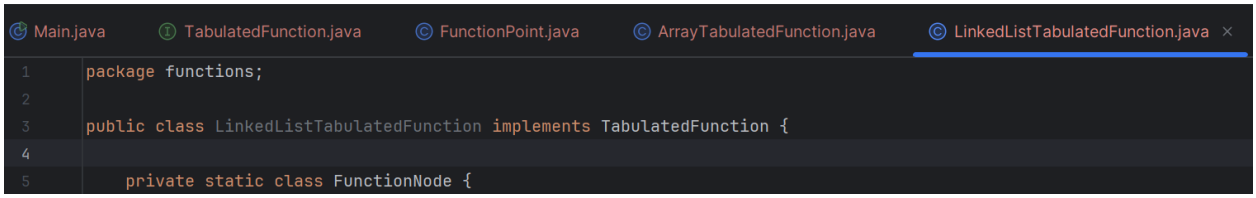

Rename
Rename class 'functions.TabulatedFunction' and its usages to:
ArrayTabulatedFunction
Search in comments and strings
Search for text occurrences
Rename tests
Rename inheritors
Rename variables
Scope:
Project Files
Refactor
Preview
Cancel


New Java Class
TabulatedFunction
Class
Interface
Record
Enum
Annotation
Exception
Compact source file

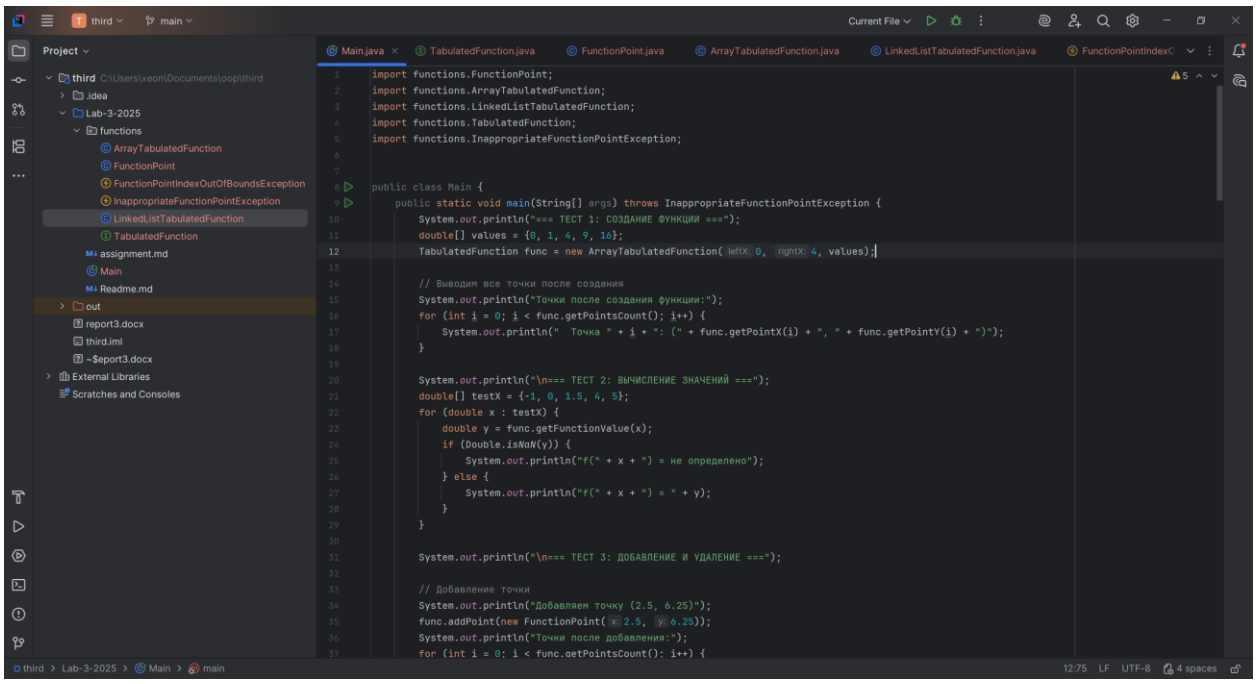


Изменил объявление класса





В main добавил импорты, throws и изменил объявления переменных на тип интерфейса



```
import functions.FunctionPoint;
import functions.ArrayTabulatedFunction;
import functions.LinkedListTabulatedFunction;
import functions.TabulatedFunction;
import functions.InappropriateFunctionPointException;


public class Main {
    public static void main(String[] args) throws InappropriateFunctionPointException {
        System.out.println("=== ТЕСТ 1: СОЗДАНИЕ ФУНКЦИИ ===");
        double[] values = {0, 1, 4, 9, 16};
        TabulatedFunction func = new ArrayTabulatedFunction( leftX: 0,  rightX: 4, values);

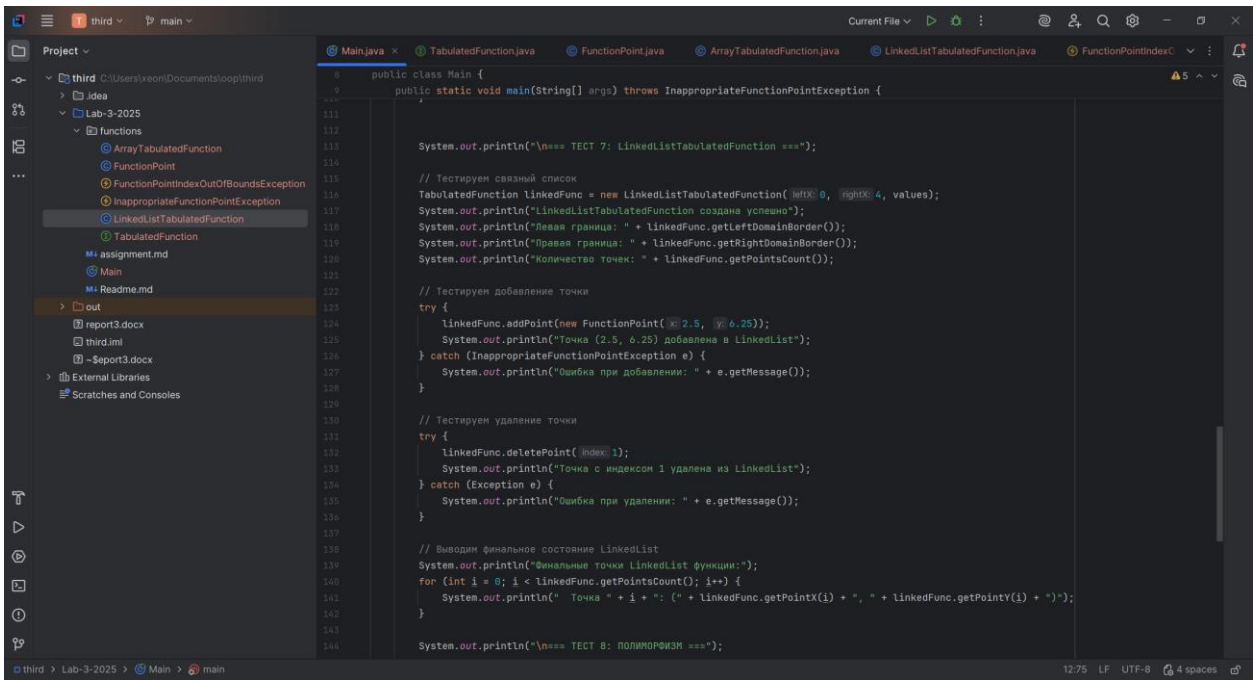
        // Выводим все точки после создания
        System.out.println("Точки после создания функции:");
        for (int i = 0; i < func.getPointsCount(); i++) {
            System.out.println("  Точка " + i + ": (" + func.getPointX(i) + ", " + func.getPointY(i) + ")");
        }

        System.out.println("\n=== ТЕСТ 2: ВЫЧИСЛЕНИЕ ЗНАЧЕНИЙ ===");
        double[] testX = {-1, 0, 1.5, 4, 5};
        for (double x : testX) {
            double y = func.getFunctionValue(x);
            if (Double.isNaN(y)) {
                System.out.println("f(" + x + ") = не определено");
            } else {
                System.out.println("f(" + x + ") = " + y);
            }
        }

        System.out.println("\n=== ТЕСТ 3: ДОБАВЛЕНИЕ И УДАЛЕНИЕ ===");

        // Добавление точки
        System.out.println("Добавляем точку (2.5, 6.25)");
        func.addPoint(new FunctionPoint( x: 2.5,  y: 6.25));
        System.out.println("Точки после добавления:");
        for (int i = 0; i < func.getPointsCount(); i++) {
```

Также добавил новые тесты



```
public class Main {
    public static void main(String[] args) throws InappropriateFunctionPointException {
        }



        System.out.println("\n=== ТЕСТ 7: LinkedListTabulatedFunction ===");

        // Тестируем связный список
        TabulatedFunction linkedFunc = new LinkedListTabulatedFunction( leftX: 0,  rightX: 4, values);
        System.out.println("LinkedListTabulatedFunction создана успешно");
        System.out.println("Левая граница: " + linkedFunc.getLeftDomainBorder());
        System.out.println("Правая граница: " + linkedFunc.getRightDomainBorder());
        System.out.println("Количество точек: " + linkedFunc.getPointsCount());

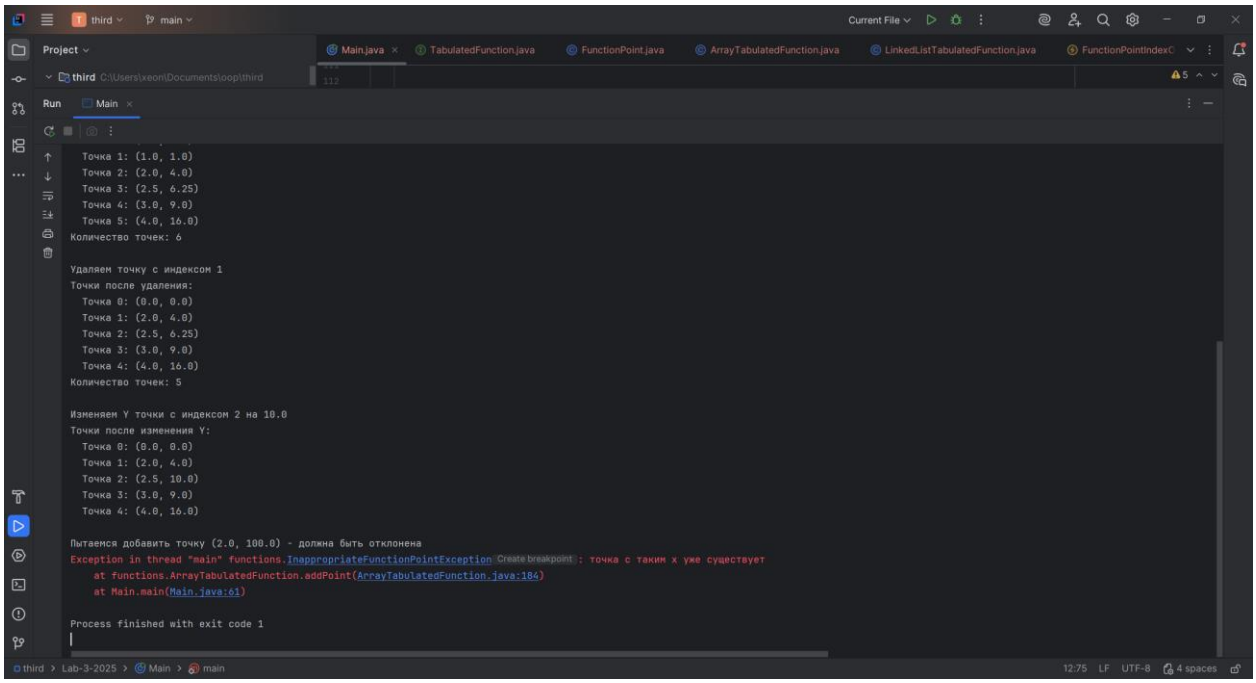
        // Тестируем добавление точки
        try {
            linkedFunc.addPoint(new FunctionPoint( x: 2.5,  y: 6.25));
            System.out.println("Точка (2.5, 6.25) добавлена в LinkedList");
        } catch (InappropriateFunctionPointException e) {
            System.out.println("Ошибка при добавлении: " + e.getMessage());
        }

        // Тестируем удаление точки
        try {
            linkedFunc.deletePoint( index: 1);
            System.out.println("Точка с индексом 1 удалена из LinkedList");
        } catch (Exception e) {
            System.out.println("Ошибка при удалении: " + e.getMessage());
        }

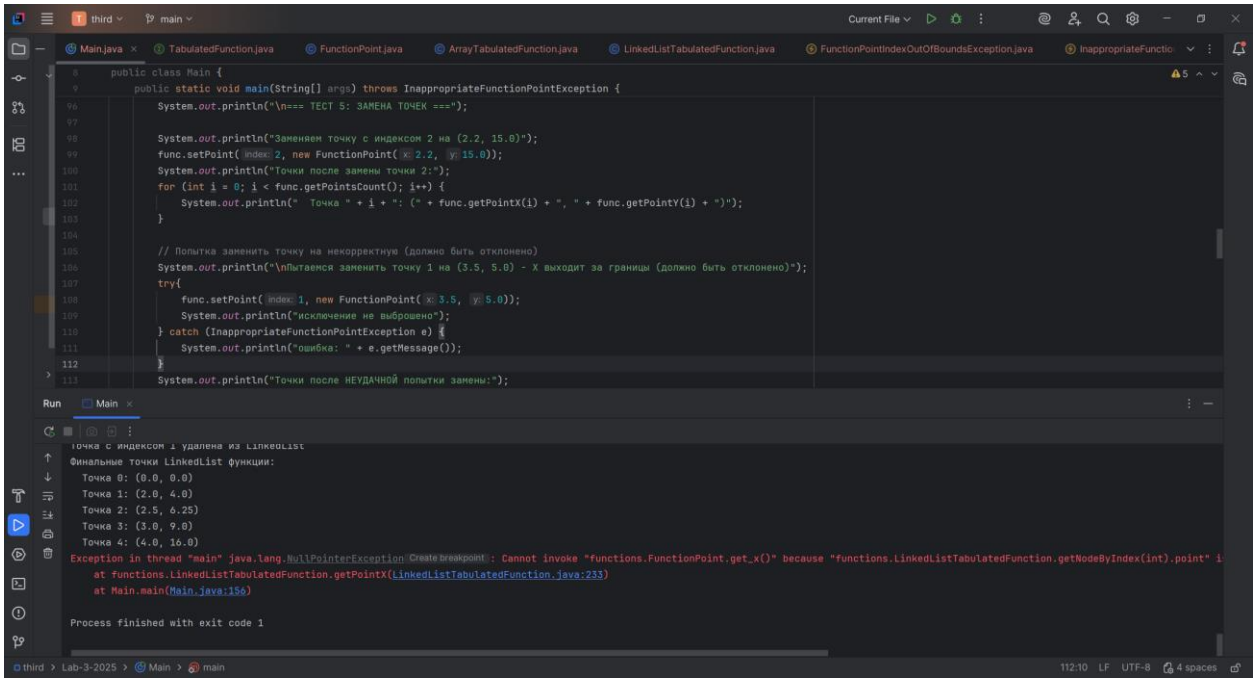
        // Выводим финальное состояние LinkedList
        System.out.println("Финальные точки LinkedList функции:");
        for (int i = 0; i < linkedFunc.getPointsCount(); i++) {
            System.out.println("  Точка " + i + ": (" + linkedFunc.getPointX(i) + ", " + linkedFunc.getPointY(i) + ")");
        }

        System.out.println("\n=== ТЕСТ 8: ПОЛИМОРФИЗМ ===");
```

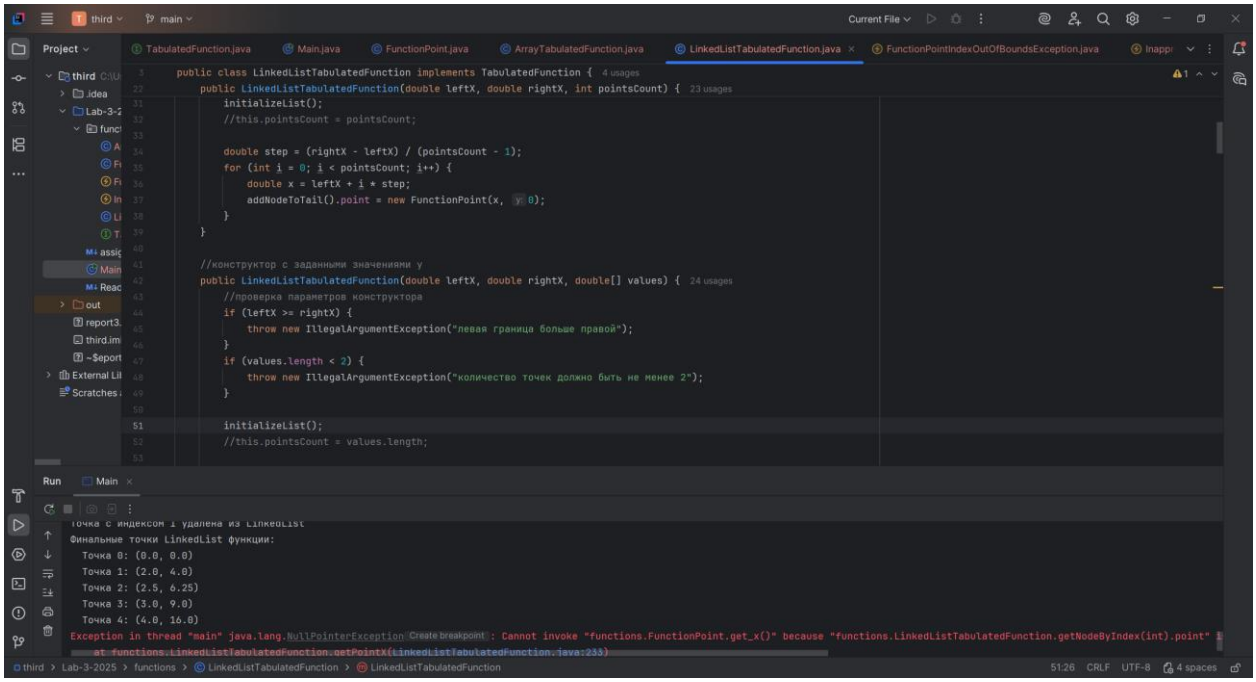
Задание 7

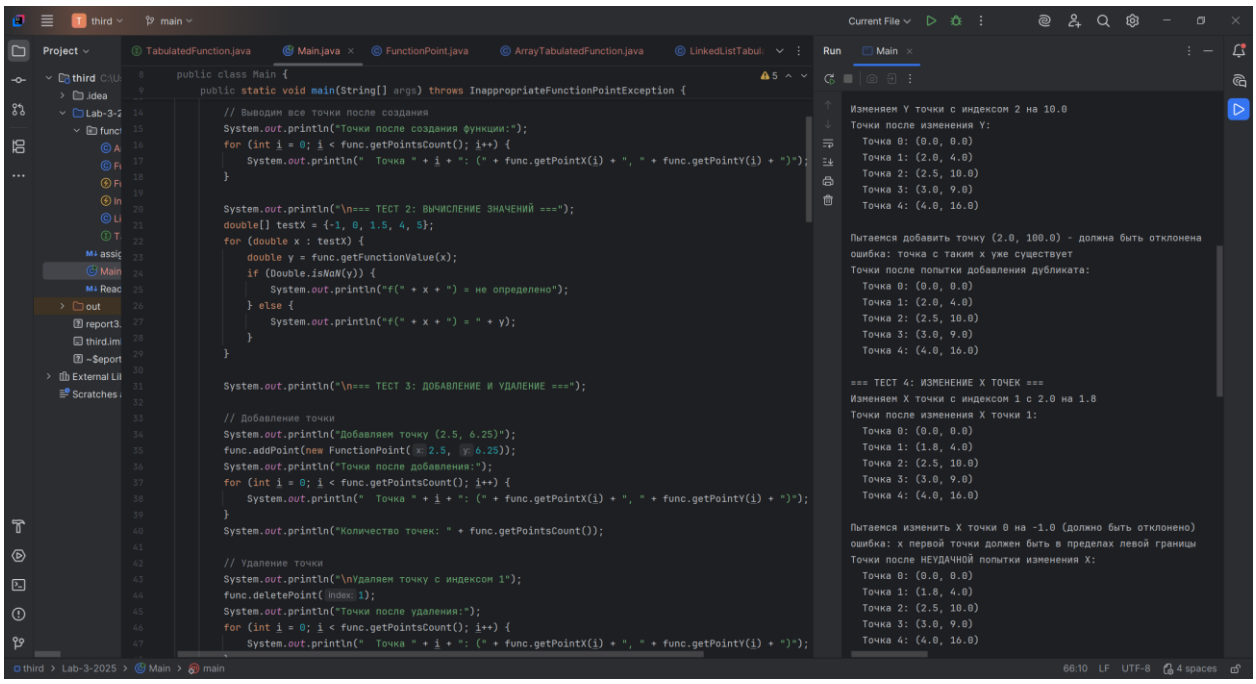


когда оборачивал всё в обработчик ошибок, то получил новую ошибку



Для решения проблемы были закомментированы строчки которые дублировали увеличение





```
// Выводим все точки после создания
System.out.println("Точки после создания функции:");
for (int i = 0; i < func.getPointsCount(); i++) {
    System.out.println("  Точка " + i + ": (" + func.getPointX(i) + ", " + func.getPointY(i) + ")");
}

System.out.println("\n=== ТЕСТ 2: ВЫЧИСЛЕНИЕ ЗНАЧЕНИЙ ===");
double[] testX = {-1, 0, 1.5, 4, 5};
for (double x : testX) {
    double y = func.getFunctionValue(x);
    if (Double.isNaN(y)) {
        System.out.println("f(" + x + ") = не определено");
    } else {
        System.out.println("f(" + x + ") = " + y);
    }
}

System.out.println("\n=== ТЕСТ 3: ДОБАВЛЕНИЕ И УДАЛЕНИЕ ===");

// Добавление точки
System.out.println("Добавляем точку (2.5, 6.25)");
func.addPoint(new FunctionPoint( x: 2.5, y: 6.25));
System.out.println("Точки после добавления:");
for (int i = 0; i < func.getPointsCount(); i++) {
    System.out.println("  Точка " + i + ": (" + func.getPointX(i) + ", " + func.getPointY(i) + ")");
}
System.out.println("Количество точек: " + func.getPointsCount());

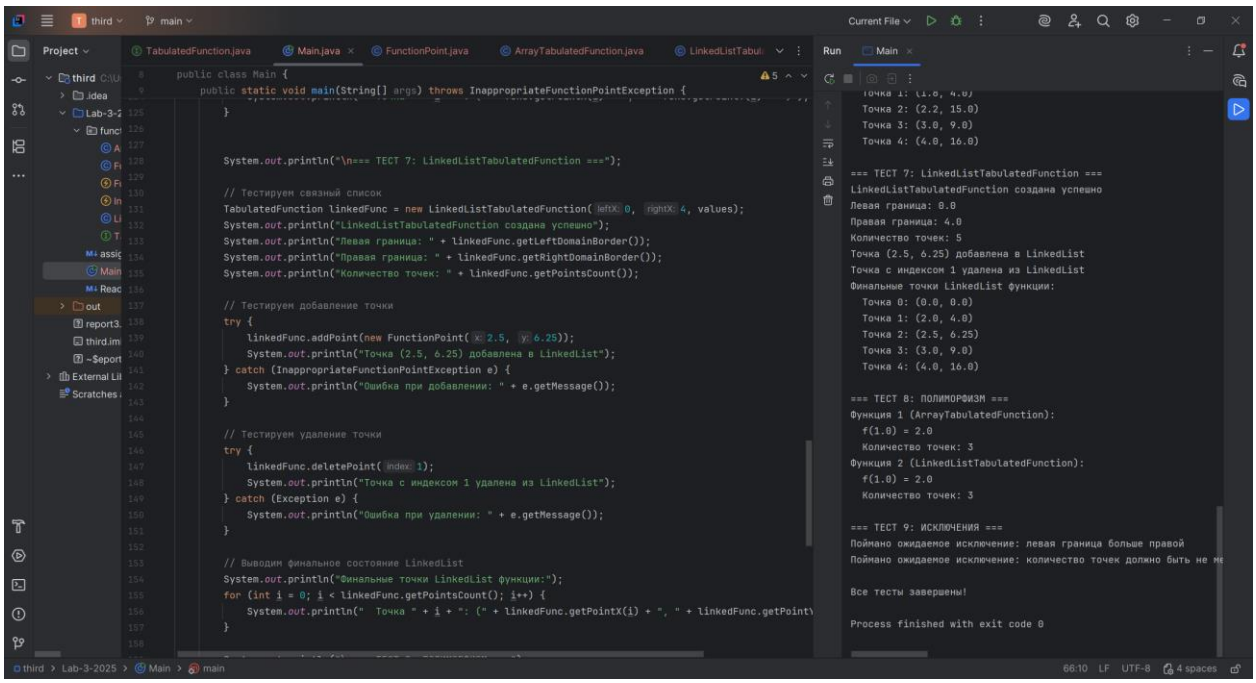
// Удаление точки
System.out.println("\nУдаляем точку с индексом 1");
func.deletePoint( index: 1);
System.out.println("Точки после удаления:");
for (int i = 0; i < func.getPointsCount(); i++) {
    System.out.println("  Точка " + i + ": (" + func.getPointX(i) + ", " + func.getPointY(i) + ")");
```

```
Изменяем Y точки с индексом 2 на 10.0
Точки после изменения Y:
  Точка 0: (0.0, 0.0)
  Точка 1: (2.0, 4.0)
  Точка 2: (2.5, 10.0)
  Точка 3: (3.0, 9.0)
  Точка 4: (4.0, 16.0)

Пытаемся добавить точку (2.0, 100.0) - должна быть отклонена
ошибка: точка с таким x уже существует
Точки после попытки добавления дубликата:
  Точка 0: (0.0, 0.0)
  Точка 1: (2.0, 4.0)
  Точка 2: (2.5, 10.0)
  Точка 3: (3.0, 9.0)
  Точка 4: (4.0, 16.0)

=== ТЕСТ 4: ИЗМЕНЕНИЕ X ТОЧЕК ===
Изменяем X точки с индексом 1 с 2.0 на 1.8
Точки после изменения X точки 1:
  Точка 0: (0.0, 0.0)
  Точка 1: (1.8, 4.0)
  Точка 2: (2.5, 10.0)
  Точка 3: (3.0, 9.0)
  Точка 4: (4.0, 16.0)

Пытаемся изменить X точки 0 на -1.0 (должно быть отклонено)
ошибка: x первой точки должен быть в пределах левой границы
Точки после НЕУДАЧНОЙ попытки изменения X:
  Точка 0: (0.0, 0.0)
  Точка 1: (1.8, 4.0)
  Точка 2: (2.5, 10.0)
  Точка 3: (3.0, 9.0)
  Точка 4: (4.0, 16.0)
```



```
        }



        System.out.println("\n=== ТЕСТ 7: LinkedListTabulatedFunction ===");

        // Тестируем связный список
        TabulatedFunction linkedFunc = new LinkedListTabulatedFunction( leftX: 0, rightX: 4, values);
        System.out.println("LinkedListTabulatedFunction создана успешно");
        System.out.println("Левая граница: " + linkedFunc.getLeftDomainBorder());
        System.out.println("Правая граница: " + linkedFunc.getRightDomainBorder());
        System.out.println("Количество точек: " + linkedFunc.getPointsCount());

        // Тестируем добавление точки
        try {
            linkedFunc.addPoint(new FunctionPoint( x: 2.5, y: 6.25));
            System.out.println("Точка (2.5, 6.25) добавлена в LinkedList");
        } catch (InappropriateFunctionPointException e) {
            System.out.println("Ошибка при добавлении: " + e.getMessage());
        }

        // Тестируем удаление точки
        try {
            linkedFunc.deletePoint( index: 1);
            System.out.println("Точка с индексом 1 удалена из LinkedList");
        } catch (Exception e) {
            System.out.println("Ошибка при удалении: " + e.getMessage());
        }

        // Выводим финальное состояние LinkedList
        System.out.println("Финальные точки LinkedList функции:");
        for (int i = 0; i < linkedFunc.getPointsCount(); i++) {
            System.out.println("  Точка " + i + ": (" + linkedFunc.getPointX(i) + ", " + linkedFunc.getPoint
        }
```

```
  Точка 2: (2.2, 15.0)
  Точка 3: (3.0, 9.0)
  Точка 4: (4.0, 16.0)

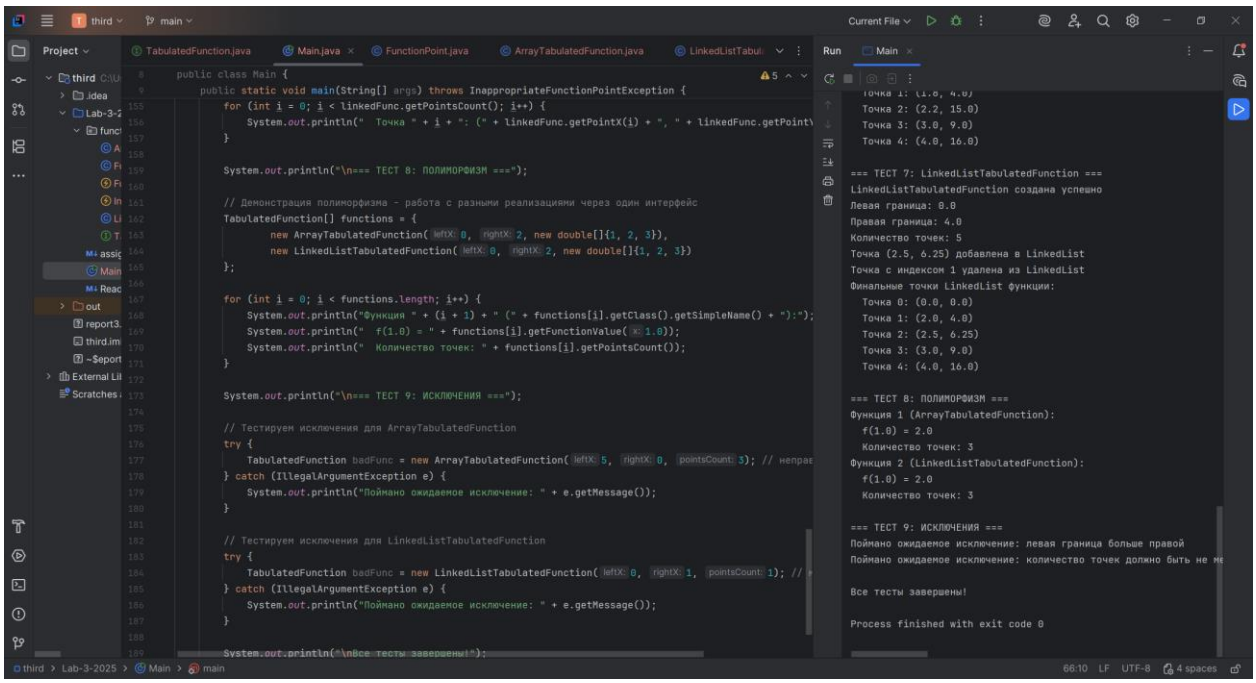
=== ТЕСТ 7: LinkedListTabulatedFunction ===
LinkedListTabulatedFunction создана успешно
Левая граница: 0.0
Правая граница: 4.0
Количество точек: 5
Точка (2.5, 6.25) добавлена в LinkedList
Точка с индексом 1 удалена из LinkedList
Финальные точки LinkedList функции:
  Точка 0: (0.0, 0.0)
  Точка 1: (2.0, 4.0)
  Точка 2: (2.5, 6.25)
  Точка 3: (3.0, 9.0)
  Точка 4: (4.0, 16.0)

=== ТЕСТ 8: ПОЛИМОРФИЗМ ===
Функция 1 (ArrayTabulatedFunction):
  f(1.0) = 2.0
  Количество точек: 3
Функция 2 (LinkedListTabulatedFunction):
  f(1.0) = 2.0
  Количество точек: 3

=== ТЕСТ 9: ИСКЛЮЧЕНИЯ ===
Поймано ожидаемое исключение: левая граница больше правой
Поймано ожидаемое исключение: количество точек должно быть не ме

Все тесты завершены!

Process finished with exit code 0
```


third
main
Current File
Project
TabulatedFunction.java
Main.java
FunctionPoint.java
ArrayTabulatedFunction.java
LinkedListTabul
Run
Main
public class Main {
public static void main(String[] args) throws InappropriateFunctionPointException {
for (int i = 0; i < linkedFunc.getPointsCount(); i++) {
System.out.println(" Точка " + i + ": (" + linkedFunc.getPointX(i) + ", " + linkedFunc.getPoint
}
System.out.println("\n=== ТЕСТ 8: ПОЛИМОРФИЗМ ===");
// Демонстрация полиморфизма - работа с разными реализациями через один интерфейс
TabulatedFunction[] functions = {
new ArrayTabulatedFunction(leftX: 0, rightX: 2, new double[]{1, 2, 3}),
new LinkedListTabulatedFunction(leftX: 0, rightX: 2, new double[]{1, 2, 3})
};
for (int i = 0; i < functions.length; i++) {
System.out.println("Функция " + (i + 1) + " (" + functions[i].getClass().getSimpleName() + "):");
System.out.println(" f(1.0) = " + functions[i].getFunctionValue(x: 1.0));
System.out.println(" Количество точек: " + functions[i].getPointsCount());
}
System.out.println("\n=== ТЕСТ 9: ИСКЛЮЧЕНИЯ ===");
// Тестируем исключения для ArrayTabulatedFunction
try {
TabulatedFunction badFunc = new ArrayTabulatedFunction(leftX: 5, rightX: 0, pointsCount: 3); // непра
} catch (IllegalArgumentException e) {
System.out.println("Поймано ожидаемое исключение: " + e.getMessage());
}
// Тестируем исключения для LinkedListTabulatedFunction
try {
TabulatedFunction badFunc = new LinkedListTabulatedFunction(leftX: 0, rightX: 1, pointsCount: 1); //
} catch (IllegalArgumentException e) {
System.out.println("Поймано ожидаемое исключение: " + e.getMessage());
}
System.out.println("\nВсе тесты завершены!");
Точка 1: (1.0, 4.0)
Точка 2: (2.2, 15.0)
Точка 3: (3.0, 9.0)
Точка 4: (4.0, 16.0)
=== ТЕСТ 7: LinkedListTabulatedFunction ===
LinkedListTabulatedFunction создана успешно
Левая граница: 0.0
Правая граница: 4.0
Количество точек: 5
Точка (2.5, 6.25) добавлена в LinkedList
Точка с индексом 1 удалена из LinkedList
Финальные точки LinkedList функции:
Точка 0: (0.0, 0.0)
Точка 1: (2.0, 4.0)
Точка 2: (2.5, 6.25)
Точка 3: (3.0, 9.0)
Точка 4: (4.0, 16.0)
=== ТЕСТ 8: ПОЛИМОРФИЗМ ===
Функция 1 (ArrayTabulatedFunction):
f(1.0) = 2.0
Количество точек: 3
Функция 2 (LinkedListTabulatedFunction):
f(1.0) = 2.0
Количество точек: 3
=== ТЕСТ 9: ИСКЛЮЧЕНИЯ ===
Поймано ожидаемое исключение: левая граница больше правой
Поймано ожидаемое исключение: количество точек должно быть не ме
Все тесты завершены!
Process finished with exit code 0
third
Lab-3-2025
Main
main
66:10
LF
UTF-8
4 spaces